A General Simulated Annealing Framework for Large Orienteering Problems with Time Windows: A Case Study of One-Day Route Planning in Suzhou

Group Name: ORacle

Members: Hao Zheng, Jiahui Qiu, Yuyao Wang, Yu Lu

Pathway: 3 – Algorithm Development for Large MIP Problems

MTH203 Introduction to Operational Research
AY 2025–26 CW2

1 Introduction

With the rapid growth of city-break tourism and mobile travel-planning services, personaliswed urban route planning has become an important operational research problem [2, 3, 10]. Travellers often expect one-day itineraries that are attractive, feasible and preference-based, while satisfying time, budget, opening-hour and transport constraints, together with comfort and route-directness requirements such as rest insertion and no-detour routing [1, 8, 9, 10, 11]. This report therefore formulates the task as an Orienteering Problem with Time Windows and Mandatory Waypoints (OPTW-MW), related to classical orienteering models.

Since exact MIP solvers may be inefficient for large urban instances and greedy methods can be trapped by early local decisions, this report proposes a Score-Maximizing Multi-Constraint Simulated Annealing algorithm (SMMC-SA) [3, 6]. The algorithm combines simulated annealing's ability to accept inferior solutions at high temperature and gradually intensify the search at low temperature with a branch-and-bound style initialisation procedure [5, 6, 7]. In this way, SMMC-SA balances score maximisation, feasibility repair and user preference, which is also implemented in a front-end system. This report, using Suzhou as a case study, presents the model, algorithm, computational results and sensitivity analysis.

2 Mathematical Model

2.1 Sets, Parameters and Decision Variables

The notation used in this study is described in Table 1.

Table 1: Notation used in the mathematical model

Symbol	**Meaning**	**Unit**
$V = \{1, \ldots, N\}$	Set of candidate attractions	–
N	Number of candidate attractions	attraction
0	Depot node; Suzhou Railway Station in the case study	–
M	Set of mandatory attractions specified by the visitor	–
X	Set of attractions excluded by the visitor	–
$A = V \setminus X$	Set of available attractions	–
S	Set of attractions selected by the model	–
$\pi = (\pi_1, \ldots, \pi_K)$	Ordered route sequence	–
π'	Candidate route generated by a neighbourhood move	–
π_k	The k-th attraction visited in the route	–
π_0, π_{K+1}	Depot at the start and end of the route	–
K	Number of visited attractions in the route	attraction
r	Step index used in the continuous activity constraint	–
ν	Visitor type, such as adult, student, child or senior	–
x_i	Binary variable; equals 1 if attraction i is visited, and 0 otherwise	–
a_i	Arrival time at attraction i	minute
B	Total travel budget	RMB

Symbol	Meaning	Unit
s_i	Base attractiveness score of attraction i	point
s'_i	Preference-adjusted score of attraction i	point
p_i	Visiting duration of attraction i	minute
$f_{\nu i}$	Ticket cost of attraction i for visitor type ν	RMB
$[e_i, l_i]$	Opening time window of attraction i	minute
τ_{ij}^{pub}	Travel time from node i to node j by public transport	minute
τ_{ij}^{taxi}	Travel time from node i to node j by taxi	minute
c_{ij}^{pub}	Travel cost from node i to node j by public transport	RMB
c_{ij}^{taxi}	Travel cost from node i to node j by taxi	RMB
τ_{ij}	Selected travel time from node i to node j in the route	minute
c_{ij}	Selected travel cost from node i to node j in the route	RMB
Δc_{ij}	Additional cost of changing leg (i, j) from public transport to taxi	RMB
T_s	Departure time of the one-day trip	minute
T_e	Latest return time to the depot on the same day	minute
$T_{\text{travel}}(\pi)$	Total travel time of route π	minute
E_r	Accumulated continuous travel-and-visit time at step r after possible rest insertion	minute
r_k	Binary rest indicator; equals 1 if a rest is required at attraction π_k, and 0 otherwise	–
T_{con}	Maximum continuous travel-and-visit time before rest is required	minute
T_{rest}	Required rest duration after exceeding the continuous activity limit	minute
w_2	Preference indicator for humanistic attractions	–
w_3	Preference indicator for natural attractions	–
w_4	Preference indicator for commercial attractions	–
P	Set of attractions whose type is selected by the user	–
Z	Objective value of the selected route	point

Let $V = \{1, \ldots, N\}$ denote the candidate attractions, and let node 0 be the depot, namely Suzhou Railway Station in the case study. The visitor may specify mandatory attractions $M \subseteq V$ and excluded attractions $X \subseteq V$, so the available set is $A = V \setminus X$. A feasible itinerary is an ordered route

$$\pi = (\pi_1, \pi_2, \ldots, \pi_K), \quad \pi_k \in A, \tag{1}$$

where K is the number of visited attractions. The route and binary selection variables are linked by

$$x_i = \begin{cases} 1, & \text{if } i = \pi_k \text{ for some } k = 1, \ldots, K, \\ 0, & \text{otherwise.} \end{cases} \tag{2}$$

Thus, an attraction appears in the route if and only if it is selected. Here, $x_i = 1$ indicates that attraction i is visited, while a_i denotes its arrival time.

Each attraction i has a base score s_i, visiting duration p_i, ticket cost $f_{\nu i}$ for visitor type ν, and

opening window $[e_i, l_i]$. For each node pair (i, j), public transport and taxi are considered, with travel times $\tau_{ij}^{pub}, \tau_{ij}^{taxi}$ and costs $c_{ij}^{pub}, c_{ij}^{taxi}$. In the realised route, the selected time and cost are denoted by τ_{ij} and c_{ij}. The trip starts at T_s and must return to the depot by T_e on the same day.

User preferences are represented by binary variables $w_2, w_3, w_4 \in \{0, 1\}$ for humanistic, natural and commercial attractions. Let P be the attractions whose categories are preferred. If $P = \varnothing$, then $s_i' = s_i$. Otherwise, the preference multiplier is

$$\lambda = \begin{cases} 1.1, & \text{if } \max\{s_j : j \notin P\} \leq \min\{s_j : j \in P\}, \\ \dfrac{\max\{s_j : j \notin P\} + 1}{\min\{s_j : j \in P\}}, & \text{if } \max\{s_j : j \notin P\} > \min\{s_j : j \in P\}. \end{cases} \tag{3}$$

The adjusted score is

$$s_i' = \begin{cases} \lambda s_i, & \text{if } i \in P, \\ s_i, & \text{otherwise.} \end{cases} \tag{4}$$

This raises the lowest-scoring preferred attraction above the highest-scoring non-preferred one, while increasing all preferred attractions proportionally. If multiple categories are selected, the same multiplier applies to all corresponding attractions, while feasibility remains governed by route constraints.

2.2 Data Source and Attraction Database

The attraction database was built from official attraction websites, OTA platforms including Ctrip, Meituan and Fliggy, and travel-sharing platforms such as Xiaohongshu and Dianping. Final values were computed by a weighted average with weights $6 : 2 : 2$, respectively. For each attraction, the database records type, score, ticket prices and recommended visiting duration, as shown in Table 2. Opening hours were obtained from Amap to define $[e_i, l_i]$, as reported in Table 3. Figure 1 illustrates the spatial distribution, with Suzhou Railway Station as the origin. Travel-time and travel-cost data are provided in Appendix Table 6. Public transport includes walking, metro and bus. Amap data are used to construct τ_{ij}^{pub}, τ_{ij}^{taxi}, c_{ij}^{pub} and c_{ij}^{taxi}.

Table 2: Attraction database for the Suzhou case study

Attraction	Type	Score	Adult Ticket (RMB)	Student Ticket (RMB)	Child Ticket (RMB)	Senior Ticket (RMB)	Duration (min)
Gate to the East	Commercial	60	0	0	0	0	40
Suzhou Center	Commercial	65	0	0	0	0	60
Humble Administrator's Garden	Humanistic	95	70	35	35	35	120
Suzhou Museum	Humanistic	90	0	0	0	0	90
Pingjiang Road	Humanistic	88	0	0	0	0	90
Lion Grove Garden	Humanistic	82	30	15	15	15	60
Guanqian Street	Commercial	70	0	0	0	0	60
Hanshan Temple	Humanistic	80	20	10	0	10	60
Fengqiao Scenic Area	Commercial	72	0	0	0	0	60
Tiger Hill	Humanistic	88	60	30	30	30	100
Lingering Garden	Humanistic	90	45	22	22	22	90
Xiyuan Temple	Humanistic	68	5	5	0	0	60
Master of the Nets Garden	Humanistic	78	30	15	15	15	60
Canglang Pavilion	Humanistic	75	15	7	7	7	60
Shantang Street	Commercial	86	0	0	0	0	90
Taihu Wetland Park	Natural	78	30	15	15	15	120
Tongli Ancient Town	Humanistic	92	100	50	50	50	180

Table 3: Opening time windows of attractions in the Suzhou case study

Attraction	Opening Time	Closing Time
Gate to the East	00:00	23:59
Suzhou Center	10:00	22:00
Humble Administrator's Garden	07:30	17:30
Suzhou Museum	09:00	17:00
Pingjiang Road	00:00	23:59
Lion Grove Garden	07:30	17:00
Guanqian Street	09:00	22:00
Hanshan Temple	07:30	17:00
Fengqiao Scenic Area	00:00	23:59
Tiger Hill	07:30	17:00
Lingering Garden	07:30	17:30
Xiyuan Temple	07:30	17:00
Master of the Nets Garden	07:30	17:00
Canglang Pavilion	07:30	17:00
Shantang Street	00:00	23:59
Taihu Wetland Park	09:00	17:00
Tongli Ancient Town	07:30	17:15

Figure 1: Simplified coordinate map of candidate attractions in Suzhou

2.3 Objective Function and Constraints

The objective maximises the total preference-adjusted score:

$$\max Z = \sum_{i \in S} s'_i, \tag{5}$$

where S is the selected attraction set.

For the ordered route $\pi = (\pi_1, \ldots, \pi_K)$, where $\pi_0 = \pi_{K+1} = 0$. Temporal feasibility is evaluated sequentially. To avoid unrealistic rest during transport, a binary rest indicator r_k is assigned to

each visited node:

$$r_k = \begin{cases} 1, & \text{if a rest is taken at node } \pi_k \text{ before moving to } \pi_{k+1}, \\ 0, & \text{otherwise,} \end{cases} \qquad k = 0, \ldots, K. \tag{6}$$

A rest is inserted either when accumulated continuous activity reaches T_{con} during a visit, or when completing the visit plus the next travel leg would exceed T_{con}. In the latter case, the traveller rests at π_k before travelling.

The arrival time of first attraction is

$$a_{\pi_1} = T_s + r_0 T_{\text{rest}} + \tau_{0,\pi_1}. \tag{7}$$

For consecutive attractions,

$$a_{\pi_{k+1}} - a_{\pi_k} = p_{\pi_k} + r_k T_{\text{rest}} + \tau_{\pi_k,\pi_{k+1}}, \quad k = 1, \ldots, K-1. \tag{8}$$

The continuous activity time is updated as

$$E_{k+1} = \begin{cases} 0, & \text{if } r_k = 1, \\ E_k + p_{\pi_k} + \tau_{\pi_k,\pi_{k+1}}, & \text{if } r_k = 0, \end{cases} \quad k = 1, \ldots, K-1. \tag{9}$$

The same-day return condition is

$$a_{\pi_K} + p_{\pi_K} + r_K T_{\text{rest}} + \tau_{\pi_K,0} \le T_e. \tag{10}$$

The selected route must also satisfy exclusion, opening-window, budget and domain constraints:

$$x_i = 0 \qquad \forall i \in X, \tag{11}$$

$$x_i = 1 \qquad \forall i \in M, \tag{12}$$

$$e_i \le a_i, \quad a_i + p_i \le l_i \qquad \forall i \in S, \tag{13}$$

$$\sum_{i \in S} f_{\nu i} + \sum_{k=0}^{K} c_{\pi_k,\pi_{k+1}} \le B, \tag{14}$$

$$x_i \in \{0, 1\} \qquad \forall i \in A, \tag{15}$$

$$a_i \ge 0 \qquad \forall i \in S. \tag{16}$$

These constraints respectively exclude user-specified attractions, ensure visits occur within opening hours, impose the total budget, and define variable domains.

3 Solution Method

3.1 Initial Solution Method

The proposed Score-Maximising Multi-Constraint Simulated Annealing algorithm (SMMC-SA) first constructs a feasible high-score initial route and then improves it through stochastic neighbourhood search. Simulated annealing is adopted because it can accept worse solutions at high

temperature to escape local optima and becomes more selective as temperature decreases [4, 6, 14].

The initial candidate set is score-oriented. If there are more than four mandatory attractions, only mandatory attractions are included:

$$C_0 = M. \tag{17}$$

Otherwise, the highest-ranked optional attractions are added:

$$C_0 = M \cup \mathrm{Top}_{4-|M|}\{i \in A \setminus M\}. \tag{18}$$

Ranking is based on s'_i. Since C_0 is small, a branch-and-bound style exact ordering subroutine determines the best visiting order. It builds routes step by step and prunes branches that violate constraints or cannot improve the incumbent. If several feasible orders have the same score, the one with shorter travel time is chosen.

All travel legs initially use public transport. If the route exceeds the daily horizon, non-mandatory attractions are removed one by one, beginning with the lowest objective contribution. After each removal, the route is reordered and re-evaluated. If only mandatory attractions remain and the route is still infeasible in time, taxi replacement is considered. For each public-transport leg (i, j), the additional taxi cost is

$$\Delta c_{ij} = c_{ij}^{taxi} - c_{ij}^{pub}. \tag{19}$$

The leg with the smallest positive Δc_{ij} is converted first, and time and budget feasibility are checked after each replacement. If taxi use violates the budget, the instance is infeasible. Once time feasibility is achieved, the budget is checked. If the budget is still exceeded, non-mandatory attractions are removed from the highest ticket cost downward. If no removable attraction remains, the instance is infeasible and mandatory attractions must be reselected.

3.2 Simulated Annealing Improvement Procedure

After initialisation, five neighbourhood operators are applied: transport-mode change with probability 20%, insertion with 30%, 2-opt with 20%, relocation with 15%, and replacement for non-mandatory attractions with 15%. These operators jointly modify transport choice, attraction selection and visit order, enabling both local adjustment and structural exploration. For 2-opt, a no-detour rule accepts a reversed route π' only if

$$T_{\mathrm{travel}}(\pi') \le T_{\mathrm{travel}}(\pi). \tag{20}$$

Otherwise, the 2-opt move is rejected directly. This prevents the algorithm from accepting a reversed route that increases total travel time without improving route quality. These operators are standard in vehicle routing and orienteering heuristics and have been effectively combined within SA frameworks for large-scale instances [7].

Each candidate route is evaluated with time windows, budget and rest insertion. The Metropolis rule is

$$P(\mathrm{accept}) = \begin{cases} 1, & \Delta Z \ge 0, \\ \exp(\Delta Z / T), & \Delta Z < 0, \end{cases} \tag{21}$$

where $\Delta Z = Z_{\text{new}} - Z_{\text{current}}$. The temperature follows geometric cooling,

$$T^{(k+1)} = \alpha T^{(k)}. \tag{22}$$

The algorithm stops when $T \le T_{\min}$, or when the best solution has not improved for 50 temperature stages.

Algorithm SMMC-SA for OPTW-MW

Require: Candidate attraction set V, available set $A = V \setminus X$, mandatory set M, excluded set X, budget B, time horizon $[T_s, T_e]$, preference indicators w_2, w_3, w_4, rest parameters $T_{\text{con}}, T_{\text{rest}}$, SA parameters $T_0, T_{\min}, \alpha, L$

Ensure: Best feasible route π^* and objective value Z^*, or infeasibility message

Construct the preferred attraction set P from w_2, w_3, w_4
if $P = \varnothing$ **then**
    Set $s'_i \leftarrow s_i$ for all $i \in A$
else
    Compute the preference multiplier

$$\lambda = \begin{cases} 1.1, & \text{if } \max\{s_j : j \notin P\} \le \min\{s_j : j \in P\}, \\ \dfrac{\max\{s_j : j \notin P\} + 1}{\min\{s_j : j \in P\}}, & \text{otherwise} \end{cases}$$

    Set $s'_i \leftarrow \lambda s_i$ for $i \in P$, and $s'_i \leftarrow s_i$ otherwise
end if
Build the initial candidate set C_0
if $|M| > 4$ **then**
    $C_0 \leftarrow M$
else
    $C_0 \leftarrow M \cup \text{Top}_{4-|M|}\{i \in A \setminus M\}$ ranked by s'_i
end if
Set the transport mode of all route legs to public transport
$\pi_0 \leftarrow$ ExactOrder(C_0) by maximizing adjusted score and then minimizing travel time
Link route and selection variables: set $x_i = 1$ if $i = \pi_k$ for some k, and $x_i = 0$ otherwise
Evaluate π_0 with time windows, budget, depot return, and rest insertion using ρ_k
while π_0 exceeds the time horizon and there exists at least one non-mandatory attraction in π_0 **do**
    Remove the non-mandatory attraction with the lowest objective contribution
    Reorder π_0 using ExactOrder
    Re-evaluate time feasibility, budget feasibility and rest insertion
end while
while π_0 exceeds the time horizon and all remaining attractions are mandatory **do**
    For each public-transport leg (i, j) in π_0, compute

$$\Delta c_{ij} = c_{ij}^{taxi} - c_{ij}^{pub}$$

    Select the leg with the smallest positive Δc_{ij}
    Change this leg from public transport to taxi

Re-evaluate total travel time and total cost
if π_0 exceeds the budget **then**
return infeasible and ask the user to relax the input requirements
end if
end while
if π_0 still exceeds the time horizon **then**
return infeasible and ask the user to relax the input requirements
end if
while π_0 exceeds the budget and there exists at least one non-mandatory attraction in π_0 **do**
Remove the non-mandatory attraction with the highest ticket cost
Reorder π_0 using ExactOrder
Re-evaluate total cost, time feasibility and rest insertion
end while
if π_0 still exceeds the budget **then**
return infeasible and ask the user to relax the input requirements
end if
$\pi_{\text{cur}} \leftarrow \pi_0$
$Z_{\text{cur}} \leftarrow Z(\pi_0)$
$\pi^* \leftarrow \pi_{\text{cur}}$
$Z^* \leftarrow Z_{\text{cur}}$
$T \leftarrow T_0$
while $T > T_{\min}$ and stopping criterion is not met **do**
for $\ell = 1$ to L **do**
Generate a candidate route π' using one neighbourhood operator:
transport-mode change (20%), insertion (30%), 2-opt (20%), relocation (15%), or replacement for non-mandatory attractions (15%)
if π' violates exclusion or duplicate-visit constraints **then**
continue
end if
if π' is generated by a 2-opt move and $T_{\text{travel}}(\pi') > T_{\text{travel}}(\pi_{\text{cur}})$ **then**
Reject π' by the no-detour rule
continue
end if
Evaluate π' with time windows, budget, depot return and rest insertion
Compute $Z' = Z(\pi')$ and $\Delta Z \leftarrow Z' - Z_{\text{cur}}$
if $\Delta Z \geq 0$ **then**
$\pi_{\text{cur}} \leftarrow \pi'$
$Z_{\text{cur}} \leftarrow Z'$
else if $U(0,1) < \exp(\Delta Z/T)$ **then**
$\pi_{\text{cur}} \leftarrow \pi'$
$Z_{\text{cur}} \leftarrow Z'$
end if
if $Z_{\text{cur}} > Z^*$ and π_{cur} is feasible **then**
$\pi^* \leftarrow \pi_{\text{cur}}$
$Z^* \leftarrow Z_{\text{cur}}$
end if

71: **end for**
72: $T \leftarrow \alpha T$
73: **end while**
74: **return** π^*, Z^*

4 Computational Results

4.1 Parameter Tuning

Parameter tuning was conducted using Case 1 in Table 5. One SA hyperparameter was varied at a time, while all other settings were fixed. The tuning results are shown in Table 4 and Figure 2.

Table 4: Sensitivity analysis for Case 1

Parameter	Value	Z	Cost (RMB)	Stops
T_0	50	696.8	204	7
T_0	200	720.8	130	7
T_0	500	663.2	222	7
T_0	800	720.8	181	7
T_0	1200	720.8	200	7
T_0	2000	720.8	171	7
α	0.70	605.6	190	6
α	0.80	624.8	178	6
α	0.85	696.8	186	7
α	0.90	696.8	215	7
α	0.93	720.8	181	7
α	0.95	696.8	211	7
α	0.98	661.2	247	7
L	10	663.2	188	7
L	20	685.6	185	7
L	30	698.8	213	7
L	40	720.8	181	7
L	60	680.6	187	7
L	80	720.8	211	7
L	100	720.8	190	7

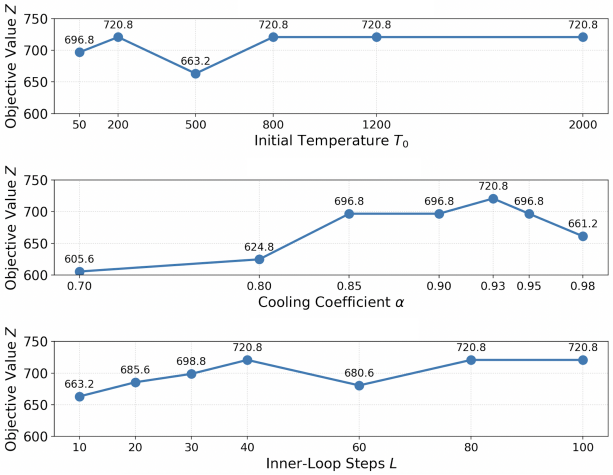

Figure 2: Curves for T_0, α, and L in Case 1

Results demonstrate that SMMC-SA is stable under different simulated annealing configurations. Various initial temperatures T_0 can achieve the best objective $Z = 720.8$. The cooling factor $\alpha = 0.93$ provides the optimal performance, and $L = 40$ is chosen to balance solution quality and efficiency. The final parameters are $T_0 = 800$, $T_{\min} = 0.5$, $\alpha = 0.93$, and $L = 40$.

4.2 Result Presentation

Four scenarios were tested to examine how SMMC-SA responds to different route-planning requirements. All experiments were repeated with 30 restarts using seed 123, and solution with highest score was selected for presentation.The scenarios include a full-day trip with mandatory attractions, a budget-constrained cultural route, a short afternoon trip, and a low-budget morning trip.

Table 5: Summary of four computational scenarios

Case	Time Horizon	Budget	Mandatory Attractions	Preference	T_{rest}	Z	Cost	Average Solve Time
1	09:00–21:00	300 RMB	Tiger Hill, Xiyuan Temple	Natural + Commercial	40 min	720.8	181 RMB	83 ms
2	09:00–18:00	150 RMB	Gate to the East, Hanshan Temple, Humble Administrator's Garden	Humanistic	40 min	399.0	145 RMB	51 ms
3	13:00–16:00	200 RMB	None	Humanistic	20 min	200.9	104 RMB	44 ms
4	09:00–13:00	50 RMB	None	None	20 min	172.0	36 RMB	36 ms

Common settings: Adult ticket type; $T_{\text{con}} = 100$ min; seed = 123; 30 restarts. The SA hyperparameters are set to $T_0 = 800$, $T_{\min} = 0.5$, $\alpha = 0.93$, and $L = 40$.

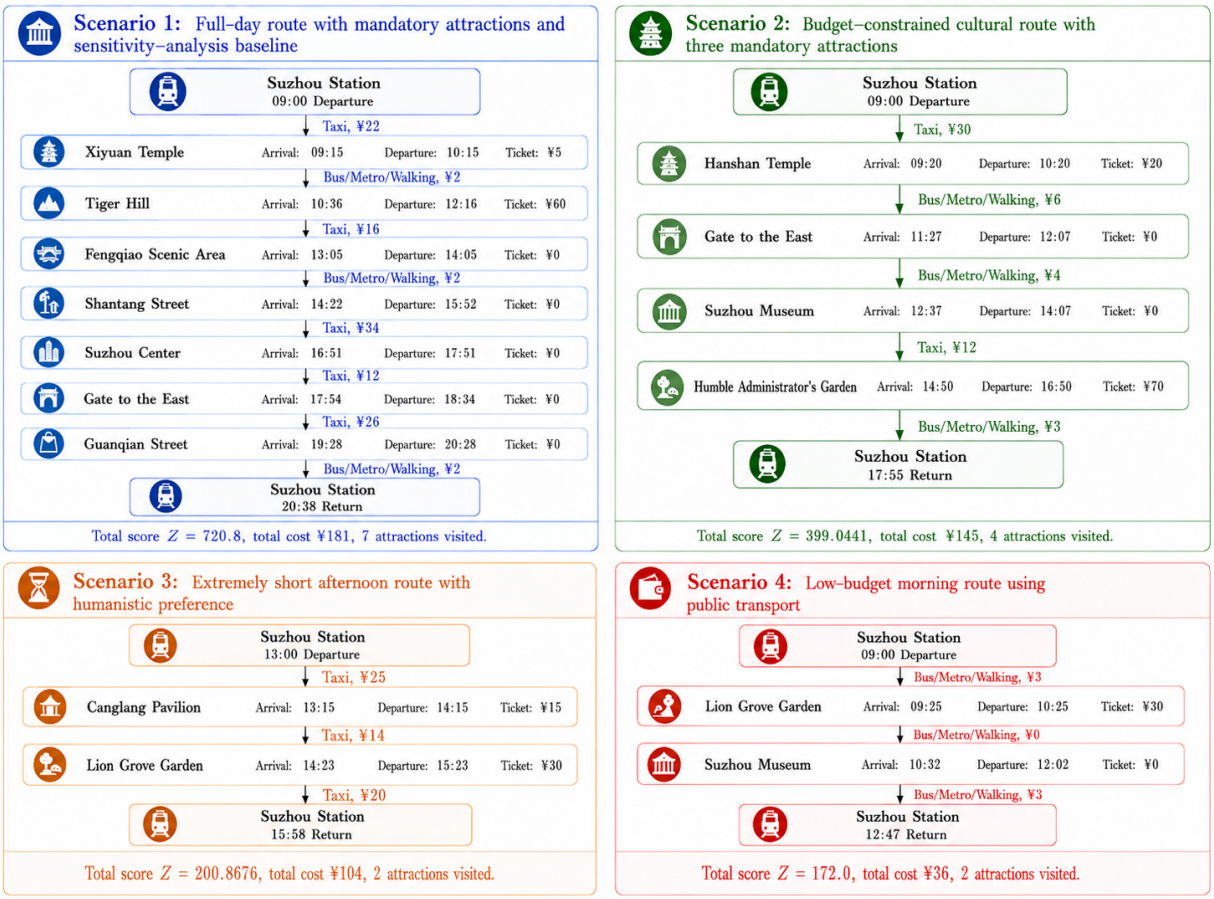

Figure 3: Route diagrams for the four test scenarios: selected attractions, transport modes, travel time, transport cost, and visiting duration

The route diagrams in Figure 3 show that SMMC-SA adapts both route length and transport mode according to the available time and budget. Case 1 contains more attractions and uses taxis on several legs to save time, while Cases 2–4 are shorter because their time or budget constraints are tighter.

4.3 Algorithm Performance

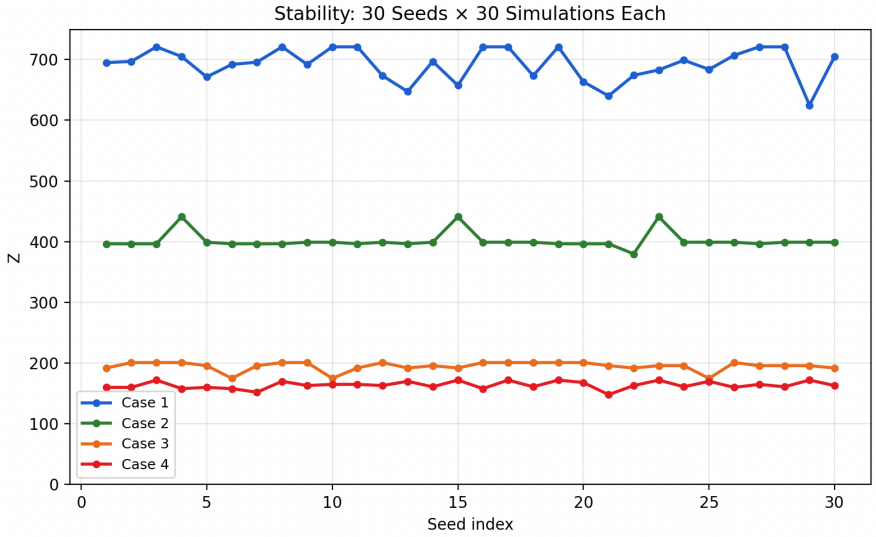

Figure 4: Stability of SMMC-SA over 30 independent seeds for the four scenarios

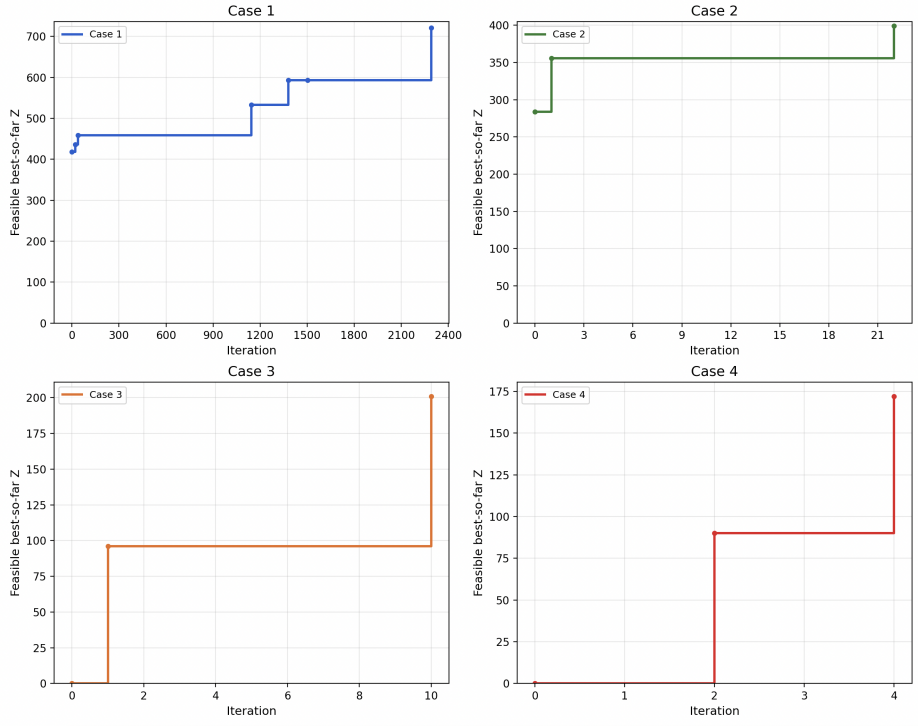

Figure 5: SMMC-SA Convergence: Feasible Best-so-far Z over Iterations

Figure 4 verifies stable outputs across 30 independent random runs. Convergence curves in Figure 5 track the best feasible objective value throughout iterations. Case 1 with loose constraints features slow, sustained optimisation owing to a broad solution space. Cases 2–4 converge rapidly and plateau early, reflecting tighter constraints restricting further objective improvement.

4.4 Sensitivity Analysis

Sensitivity analysis was conducted on budget, T_{con}, T_{rest} and T_{min}. As illustrated in Figure 6, objective value Z rises with budget and levels off after reaching a critical limit. Larger T_{con} boosts Z by cutting rest frequency and extending sightseeing time, whereas prolonged T_{rest} lowers performance due to time consumption and fewer accessible attractions. The stopping temperature T_{min} has negligible influence on the final solution. Once sufficiently small, further reduction only increases computation without altering the route, confirming the robustness of SMMC-SA.

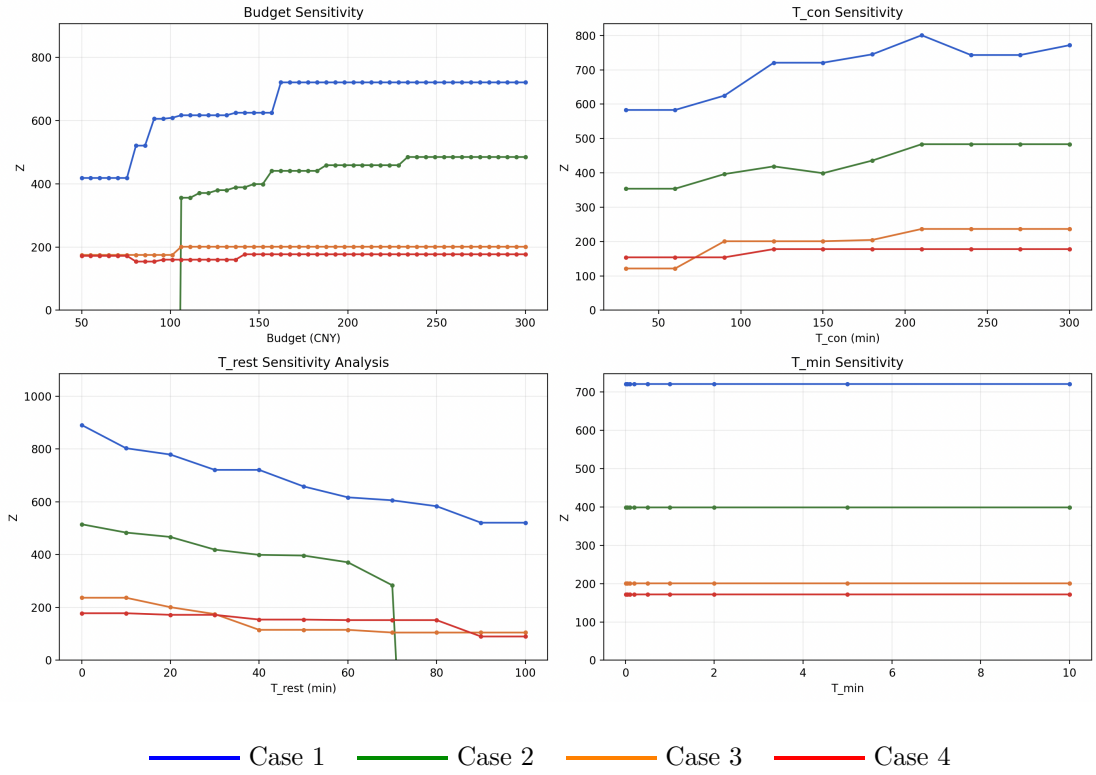

Figure 6: Sensitivity analysis of SMMC-SA under four parameters: budget, T_{con}, T_{rest}, and minimum temperature T_{min}

5 Conclusion

SMMC-SA effectively balances score maximization, time windows, budget limits and mandatory-visit requirements. The four scenarios show that the algorithm can adapt route length and transport mode to different constraint settings. The stability and sensitivity tests also indicate reasonable robustness. This framework can also be transferred to other cities by updating the attraction database and travel matrix. Its integration with a user-facing interface further demonstrates its practical value for personalized one-day itinerary planning, which is available at: `https://oracle-suzhou-travel.base44.app`. Future research directions include developing scalable versions for large-scale instances with hundreds of points of interest, extending the model to multi-day itineraries, and incorporating real-time traffic and environmental data.

Word count: 1547 words, excluding tables, equations, references and appendix.

References

[1] Choachaicharoenkul, S., Coit, D., & Wattanapongsakorn, N. (2022). Multi-objective trip planning with solution ranking based on user preference and restaurant selection. *IEEE Access*, *10*, 10688–10706. https://doi.org/10.1109/ACCESS.2022.3144855

[2] Cui, X., Wang, Z., Li, P., & Xu, Q. (2025). I-AIR: Intention-aware travel itinerary recommendation via multi-signal fusion and spatiotemporal constraints. *Journal of King Saud University – Computer and Information Sciences*, *37*, 169. https://doi.org/10.1007/s44443-025-00178-0

[3] Jewpanya, P., Nuangpirom, P., Pitjamit, S., & Nakkiew, W. (2025). Optimized travel itineraries: Combining mandatory visits and personalized activities. *Algorithms*, *18*(2), 110. https://doi.org/10.3390/a18020110

[4] Kirkpatrick, S., Gelatt, C. D., & Vecchi, M. P. (1983). Optimization by simulated annealing. *Science*, *220*(4598), 671–680. https://doi.org/10.1126/science.220.4598.671

[5] Kirac, E., Gedik, R., & Oztanriseven, F. (2023). Solving the team orienteering problem with time windows and mandatory visits using a constraint programming approach. *International Journal of Operational Research*, *46*(1), 20–42. https://doi.org/10.1504/IJOR.2023.128542

[6] Lin, S.-W. (2013). Solving the team orienteering problem using effective multi-start simulated annealing. *Applied Soft Computing*, *13*(2), 1064–1073. https://doi.org/10.1016/j.asoc.2012.09.022

[7] Lin, S.-W., & Yu, V. F. (2015). A simulated annealing heuristic for the multiconstraint team orienteering problem with multiple time windows. *Applied Soft Computing*, *37*, 632–642. https://doi.org/10.1016/j.asoc.2015.08.058

[8] Mangini, A. M., Roccotelli, M., & Rinaldi, A. (2021). A novel application based on a heuristic approach for planning itineraries of one-day tourist. *Applied Sciences*, *11*(19), 8989. https://doi.org/10.3390/app11198989

[9] Tang, Y., Wang, Z., Qu, A., Yan, Y., Wu, Z., Zhuang, D., Kai, J., Hou, K., Guo, X., Zheng, H., Luo, T., Zhao, J., Zhao, Z., & Ma, W. (2025). ITINERA: Integrating spatial optimization with large language models for open-domain urban itinerary planning. *arXiv preprint arXiv:2402.07204*. https://arxiv.org/abs/2402.07204

[10] Tenemaza, M., Luján-Mora, S., de Antonio, A., & Ramírez, J. (2020). Improving itinerary recommendations for tourists through metaheuristic algorithms: An optimization proposal. *IEEE Access*, *8*, 79003–79021. https://doi.org/10.1109/ACCESS.2020.2990348

[11] Wins, A., Barthelmes, L., Alpers, S., Becker, C., Kagerbauer, M., & Oberweis, A. (2024). Personalized day-trip planning: A TSP-TW-based multimodal multicriteria optimisation approach. *Procedia Computer Science*, *238*, 352–360. https://doi.org/10.1016/j.procs.2024.06.035

[12] Gavalas, D., Konstantopoulos, C., Mastakas, K., Pantziou, G., & Vathis, N. (2016). Efficient metaheuristics for the mixed team orienteering problem with time windows. *Algorithms*, *9*(1), 6. https://doi.org/10.3390/a9010006

[13] Behdadnia, M., & Askerzade, I. N. (2016). Simulated annealing approach for solving time dependent orienteering problem. *Communications*, 1–20.

[14] Zhan, S.-H., Lin, J., Zhang, Z.-J., & Zhong, Y.-W. (2016). List-based simulated annealing algorithm for traveling salesman problem. *Discrete Dynamics in Nature and Society*, *2016*, 1712630. https://doi.org/10.1155/2016/1712630

6 Appendix: Traffic Data

Table 6: Travel time and cost between attractions in Suzhou

From	To	Public Time (min)	Public Cost (RMB)	Taxi Time (min)	Taxi Cost (RMB)
Gate to the East	Suzhou Center	5	0	3	12
Gate to the East	Humble Administrator's Garden	35	4	22	38
Gate to the East	Suzhou Museum	30	4	23	40
Gate to the East	Pingjiang Road	19	4	18	32
Gate to the East	Lion Grove Garden	37	4	21	36
Gate to the East	Guanqian Street	23	3	14	26
Gate to the East	Hanshan Temple	27	6	24	42
Gate to the East	Fengqiao Scenic Area	30	6	25	44
Gate to the East	Tiger Hill	43	6	28	48
Gate to the East	Lingering Garden	36	4	23	40
Gate to the East	Xiyuan Temple	32	4	22	38
Gate to the East	Master of the Nets Garden	37	4	19	34
Gate to the East	Canglang Pavilion	30	4	18	32
Gate to the East	Shantang Street	25	6	19	34
Gate to the East	Taihu Wetland Park	70	8	38	92

From	To	Public Time (min)	Public Cost (RMB)	Taxi Time (min)	Taxi Cost (RMB)
Gate to the East	Tongli Ancient Town	70	7	39	102
Suzhou Center	Humble Administrator's Garden	35	4	22	38
Suzhou Center	Suzhou Museum	40	4	23	40
Suzhou Center	Pingjiang Road	33	4	18	32
Suzhou Center	Lion Grove Garden	37	4	21	36
Suzhou Center	Guanqian Street	23	3	14	26
Suzhou Center	Hanshan Temple	27	6	24	42
Suzhou Center	Fengqiao Scenic Area	30	6	25	44
Suzhou Center	Tiger Hill	43	6	28	48
Suzhou Center	Lingering Garden	36	4	23	40
Suzhou Center	Xiyuan Temple	32	4	22	38
Suzhou Center	Master of the Nets Garden	37	4	19	34
Suzhou Center	Canglang Pavilion	30	4	18	32
Suzhou Center	Shantang Street	25	6	19	34
Suzhou Center	Taihu Wetland Park	70	8	38	92
Suzhou Center	Tongli Ancient Town	70	7	39	102
Humble Administrator's Garden	Suzhou Museum	3	0	3	12
Humble Administrator's Garden	Pingjiang Road	12	0	4	12

From	To	Public Time (min)	Public Cost (RMB)	Taxi Time (min)	Taxi Cost (RMB)
Humble Administrator's Garden	Lion Grove Garden	8	0	3	12
Humble Administrator's Garden	Guanqian Street	18	0	5	12
Humble Administrator's Garden	Hanshan Temple	32	2	14	24
Humble Administrator's Garden	Fengqiao Scenic Area	36	2	15	26
Humble Administrator's Garden	Tiger Hill	28	2	13	22
Humble Administrator's Garden	Lingering Garden	24	2	9	16
Humble Administrator's Garden	Xiyuan Temple	28	2	9	16
Humble Administrator's Garden	Master of the Nets Garden	22	2	9	16
Humble Administrator's Garden	Canglang Pavilion	22	2	9	16
Humble Administrator's Garden	Shantang Street	20	2	8	14
Humble Administrator's Garden	Taihu Wetland Park	65	5	33	78
Humble Administrator's Garden	Tongli Ancient Town	55	6	34	88
Suzhou Museum	Pingjiang Road	10	0	4	12
Suzhou Museum	Lion Grove Garden	7	0	3	12
Suzhou Museum	Guanqian Street	17	0	5	12
Suzhou Museum	Hanshan Temple	33	2	14	24

From	To	Public Time (min)	Public Cost (RMB)	Taxi Time (min)	Taxi Cost (RMB)
Suzhou Museum	Fengqiao Scenic Area	37	2	15	26
Suzhou Museum	Tiger Hill	29	2	13	22
Suzhou Museum	Lingering Garden	25	2	9	16
Suzhou Museum	Xiyuan Temple	29	2	9	16
Suzhou Museum	Master of the Nets Garden	23	2	9	16
Suzhou Museum	Canglang Pavilion	23	2	9	16
Suzhou Museum	Shantang Street	21	2	8	14
Suzhou Museum	Taihu Wetland Park	66	5	33	78
Suzhou Museum	Tongli Ancient Town	56	6	34	88
Pingjiang Road	Lion Grove Garden	6	0	3	12
Pingjiang Road	Guanqian Street	14	0	0	0
Pingjiang Road	Hanshan Temple	26	6	14	24
Pingjiang Road	Fengqiao Scenic Area	29	6	15	26
Pingjiang Road	Tiger Hill	28	7	13	22
Pingjiang Road	Lingering Garden	32	4	9	16
Pingjiang Road	Xiyuan Temple	28	4	9	16
Pingjiang Road	Master of the Nets Garden	11	0	4	12
Pingjiang Road	Canglang Pavilion	16	0	5	12
Pingjiang Road	Shantang Street	20	5	8	14

From	To	Public Time (min)	Public Cost (RMB)	Taxi Time (min)	Taxi Cost (RMB)
Pingjiang Road	Taihu Wetland Park	69	8	33	78
Pingjiang Road	Tongli Ancient Town	50	6	34	88
Lion Grove Garden	Guanqian Street	13	0	4	12
Lion Grove Garden	Hanshan Temple	31	2	14	24
Lion Grove Garden	Fengqiao Scenic Area	35	2	15	26
Lion Grove Garden	Tiger Hill	27	2	13	22
Lion Grove Garden	Lingering Garden	23	2	9	16
Lion Grove Garden	Xiyuan Temple	27	2	9	16
Lion Grove Garden	Master of the Nets Garden	21	2	8	14
Lion Grove Garden	Canglang Pavilion	21	2	8	14
Lion Grove Garden	Shantang Street	19	2	8	14
Lion Grove Garden	Taihu Wetland Park	64	5	33	78
Lion Grove Garden	Tongli Ancient Town	54	6	34	88
Guanqian Street	Hanshan Temple	24	5	11	18
Guanqian Street	Fengqiao Scenic Area	27	5	12	20
Guanqian Street	Tiger Hill	27	7	11	18
Guanqian Street	Lingering Garden	30	3	9	16
Guanqian Street	Xiyuan Temple	26	3	9	16
Guanqian Street	Master of the Nets Garden	18	2	4	12

From	To	Public Time (min)	Public Cost (RMB)	Taxi Time (min)	Taxi Cost (RMB)
Guanqian Street	Canglang Pavilion	13	0	4	12
Guanqian Street	Shantang Street	19	5	7	14
Guanqian Street	Taihu Wetland Park	68	7	32	76
Guanqian Street	Tongli Ancient Town	40	3	33	86
Hanshan Temple	Fengqiao Scenic Area	9	0	3	12
Hanshan Temple	Tiger Hill	23	2	9	16
Hanshan Temple	Lingering Garden	17	2	4	12
Hanshan Temple	Xiyuan Temple	13	2	4	12
Hanshan Temple	Master of the Nets Garden	33	4	14	24
Hanshan Temple	Canglang Pavilion	26	4	14	24
Hanshan Temple	Shantang Street	17	2	8	14
Hanshan Temple	Taihu Wetland Park	55	3	23	54
Hanshan Temple	Tongli Ancient Town	57	6	32	86
Fengqiao Scenic Area	Tiger Hill	23	2	9	16
Fengqiao Scenic Area	Lingering Garden	17	2	4	12
Fengqiao Scenic Area	Xiyuan Temple	13	2	4	12
Fengqiao Scenic Area	Master of the Nets Garden	33	4	14	24
Fengqiao Scenic Area	Canglang Pavilion	26	4	14	24
Fengqiao Scenic Area	Shantang Street	17	2	8	14

From	To	Public Time (min)	Public Cost (RMB)	Taxi Time (min)	Taxi Cost (RMB)
Fengqiao Scenic Area	Taihu Wetland Park	55	3	23	54
Fengqiao Scenic Area	Tongli Ancient Town	57	6	32	86
Tiger Hill	Lingering Garden	17	2	8	14
Tiger Hill	Xiyuan Temple	21	2	8	14
Tiger Hill	Master of the Nets Garden	29	4	14	24
Tiger Hill	Canglang Pavilion	29	4	14	24
Tiger Hill	Shantang Street	13	2	4	12
Tiger Hill	Taihu Wetland Park	45	3	19	46
Tiger Hill	Tongli Ancient Town	57	6	31	82
Lingering Garden	Xiyuan Temple	9	0	3	12
Lingering Garden	Master of the Nets Garden	25	4	13	22
Lingering Garden	Canglang Pavilion	25	4	13	22
Lingering Garden	Shantang Street	13	2	4	12
Lingering Garden	Taihu Wetland Park	45	3	19	46
Lingering Garden	Tongli Ancient Town	52	6	31	82
Xiyuan Temple	Master of the Nets Garden	25	4	13	22
Xiyuan Temple	Canglang Pavilion	25	4	13	22

From	To	Public Time (min)	Public Cost (RMB)	Taxi Time (min)	Taxi Cost (RMB)
Xiyuan Temple	Shantang Street	13	2	4	12
Xiyuan Temple	Taihu Wetland Park	42	3	19	46
Xiyuan Temple	Tongli Ancient Town	52	6	31	82
Master of the Nets Garden	Canglang Pavilion	9	0	4	12
Master of the Nets Garden	Shantang Street	27	4	13	22
Master of the Nets Garden	Taihu Wetland Park	67	7	32	76
Master of the Nets Garden	Tongli Ancient Town	48	5	29	78
Canglang Pavilion	Shantang Street	27	4	13	22
Canglang Pavilion	Taihu Wetland Park	67	7	32	76
Canglang Pavilion	Tongli Ancient Town	48	5	29	78
Shantang Street	Taihu Wetland Park	55	3	22	52
Shantang Street	Tongli Ancient Town	47	5	31	82
Taihu Wetland Park	Tongli Ancient Town	75	8	38	98
Suzhou Station	Gate to the East	40	4	25	35
Suzhou Station	Suzhou Center	40	4	25	35
Suzhou Station	Humble Administrator's Garden	25	3	15	20

From	To	Public Time (min)	Public Cost (RMB)	Taxi Time (min)	Taxi Cost (RMB)
Suzhou Station	Suzhou Museum	25	3	15	20
Suzhou Station	Pingjiang Road	30	3	12	18
Suzhou Station	Lion Grove Garden	25	3	15	20
Suzhou Station	Guanqian Street	10	2	8	15
Suzhou Station	Hanshan Temple	35	4	20	30
Suzhou Station	Fengqiao Scenic Area	35	4	20	30
Suzhou Station	Tiger Hill	30	3	18	28
Suzhou Station	Lingering Garden	20	3	15	22
Suzhou Station	Xiyuan Temple	20	3	15	22
Suzhou Station	Master of the Nets Garden	35	4	18	28
Suzhou Station	Canglang Pavilion	25	3	15	25
Suzhou Station	Shantang Street	10	2	8	15
Suzhou Station	Taihu Wetland Park	70	6	40	85
Suzhou Station	Tongli Ancient Town	60	5	45	95

7 Appendix: Implementation Notes

The implementation represents each attraction as a record containing its ID, name, type, score, visiting duration, ticket costs and opening time. Travel data are stored in a symmetric matrix containing public-transport and taxi options. The route evaluator simulates the itinerary sequentially from the depot, updates arrival and departure times, inserts waiting time if an attraction has not opened, and rejects visits that would end after closing time. It also accumulates ticket and transport costs and inserts rests whenever the continuous activity limit is exceeded.

During initialization, the transport mode of every route leg is first set to public transport. If the route exceeds the total travel time, non-mandatory attractions are removed one by one and the

route is recalculated after each removal. If no non-mandatory attractions remain and the route is still too long, the algorithm changes public-transport legs to taxi, starting from the leg with the smallest additional taxi cost. If the resulting route exceeds the budget, or if the budget is still exceeded after all removable non-mandatory attractions have been deleted, the instance is declared infeasible and the user is asked to re-enter the requirements.

The five neighbourhood operators are randomly selected using fixed probabilities: transport-mode change 20%, insert 30%, 2-opt 20%, relocate 15% and replace 15%. The default convergence rule stops the search after 50 temperature stages without improvement.

8 Appendix: Artificial Intelligence Usage Statement

The overall solution strategy, mathematical model, and SMMC-SA algorithm framework were fully designed by the team members without artificial intelligence (AI) assistance. Once the algorithm logic was completely determined, we provided detailed step-by-step instructions to AI to produce an initial Python implementation code of the SMMC-SA algorithm. The AI-generated code was then reviewed line-by-line by the team. Then, errors, missing constraints and inefficiencies were identified through testing, and the code was revised through follow-up prompts. After several iterations, the final code was verified by us to be correct and fully consistent with the proposed algorithm.

For the front-end prototype accessible at `https://oracle-suzhou-travel.base44.app`, we provided the final Python code from the Appendix "Python Code Implementation" to an AI tool and instructed it to generate a corresponding web interface based on that code. The AI-generated front-end code was then tested by the team. We corrected display and logic issues, refined the design, and ensured the interface reliably produces personalised itineraries from user inputs based on our designed algorithm.

9 Appendix: Python Code Implementation

The complete Python implementation of the proposed SMMC-SA algorithm is provided below.

Listing 1: Full implementation of SMMC-SA

```
import math, random, time
from datetime import date as _date

# 1. Attraction Database
SUZHOU_ATTRACTIONS = [
    # id, name, type, score, baseTime(min),
    # ticket{adult,student,child,senior}(CNY), openTime, closeTime
    {"id": "dfjz",  "name": "Gate to the East",              "type": "commercial", "score"
        : 60, "baseTime": 40,
     "ticket": {"adult": 0,   "student": 0,  "child": 0,  "senior": 0},
     "openTime": "00:00", "closeTime": "23:59"},
    {"id": "szzx",  "name": "Suzhou Center",                 "type": "commercial", "score"
        : 65, "baseTime": 60,
     "ticket": {"adult": 0,   "student": 0,  "child": 0,  "senior": 0},
     "openTime": "10:00", "closeTime": "22:00"},
    {"id": "zhzy",  "name": "Humble Administrator's Garden", "type": "humanistic", "score"
        : 95, "baseTime": 120,
     "ticket": {"adult": 70,  "student": 35, "child": 35, "senior": 35},
     "openTime": "07:30", "closeTime": "17:30"},
```

```
    {"id": "szbwg", "name": "Suzhou Museum",                "type": "humanistic", "score": 90, "baseTime": 90,
     "ticket": {"adult": 0,   "student": 0,  "child": 0,  "senior": 0},
     "openTime": "09:00", "closeTime": "17:00"},
    {"id": "pjl",   "name": "Pingjiang Road",               "type": "humanistic", "score": 88, "baseTime": 90,
     "ticket": {"adult": 0,   "student": 0,  "child": 0,  "senior": 0},
     "openTime": "00:00", "closeTime": "23:59"},
    {"id": "szl",   "name": "Lion Grove Garden",            "type": "humanistic", "score": 82, "baseTime": 60,
     "ticket": {"adult": 30,  "student": 15, "child": 15, "senior": 15},
     "openTime": "07:30", "closeTime": "17:00"},
    {"id": "clq",   "name": "Guanqian Street",              "type": "commercial", "score": 70, "baseTime": 60,
     "ticket": {"adult": 0,   "student": 0,  "child": 0,  "senior": 0},
     "openTime": "09:00", "closeTime": "22:00"},
    {"id": "hss",   "name": "Hanshan Temple",               "type": "humanistic", "score": 80, "baseTime": 60,
     "ticket": {"adult": 20,  "student": 10, "child": 0,  "senior": 10},
     "openTime": "07:30", "closeTime": "17:00"},
    {"id": "fqjq",  "name": "Fengqiao Scenic Area",         "type": "commercial", "score": 72, "baseTime": 60,
     "ticket": {"adult": 0,   "student": 0,  "child": 0,  "senior": 0},
     "openTime": "00:00", "closeTime": "23:59"},
    {"id": "hq",    "name": "Tiger Hill",                   "type": "humanistic", "score": 88, "baseTime": 100,
     "ticket": {"adult": 60,  "student": 30, "child": 30, "senior": 30},
     "openTime": "07:30", "closeTime": "17:00"},
    {"id": "ly",    "name": "Lingering Garden",             "type": "humanistic", "score": 90, "baseTime": 90,
     "ticket": {"adult": 45,  "student": 22, "child": 22, "senior": 22},
     "openTime": "07:30", "closeTime": "17:30"},
    {"id": "xyj",   "name": "Xiyuan Temple",                "type": "humanistic", "score": 68, "baseTime": 60,
     "ticket": {"adult": 5,   "student": 5,  "child": 0,  "senior": 0},
     "openTime": "07:30", "closeTime": "17:00"},
    {"id": "wsy",   "name": "Master of the Nets Garden",    "type": "humanistic", "score": 78, "baseTime": 60,
     "ticket": {"adult": 30,  "student": 15, "child": 15, "senior": 15},
     "openTime": "07:30", "closeTime": "17:00"},
    {"id": "clf",   "name": "Canglang Pavilion",            "type": "humanistic", "score": 75, "baseTime": 60,
     "ticket": {"adult": 15,  "student": 7,  "child": 7,  "senior": 7},
     "openTime": "07:30", "closeTime": "17:00"},
    {"id": "stj",   "name": "Shantang Street",              "type": "commercial", "score": 86, "baseTime": 90,
     "ticket": {"adult": 0,   "student": 0,  "child": 0,  "senior": 0},
     "openTime": "00:00", "closeTime": "23:59"},
    {"id": "thlgy", "name": "Taihu Wetland Park",           "type": "natural",    "score": 78, "baseTime": 120,
     "ticket": {"adult": 30,  "student": 15, "child": 15, "senior": 15},
     "openTime": "09:00", "closeTime": "17:00"},
    {"id": "tllgz", "name": "Tongli Ancient Town",          "type": "humanistic", "score": 92, "baseTime": 180,
     "ticket": {"adult": 100, "student": 50, "child": 50, "senior": 50},
     "openTime": "07:30", "closeTime": "17:15"},
]

HUMANISTIC_TYPES = {"humanistic"}
NATURAL_TYPES    = {"natural"}
COMMERCIAL_TYPES = {"commercial"}

N          = len(SUZHOU_ATTRACTIONS)
ATT_BY_ID = {a["id"]: a for a in SUZHOU_ATTRACTIONS}

# ============================================================
```

```
# 2. Transport Matrix
#    Each entry: [from, to, public_time(min), public_cost(CNY),
#                           taxi_time(min),   taxi_cost(CNY)]

RAW_TRAFFIC = [
    ["dfjz","szzx",5,0,3,12],     ["dfjz","zhzy",35,4,22,38],   ["dfjz","szbwg"
        ,30,4,23,40],
    ["dfjz","pjl",19,4,18,32],    ["dfjz","szl",37,4,21,36],    ["dfjz","clq",23,3,14,26],
    ["dfjz","hss",27,6,24,42],    ["dfjz","fqjq",30,6,25,44],   ["dfjz","hq",43,6,28,48],
    ["dfjz","ly",36,4,23,40],     ["dfjz","xyj",32,4,22,38],    ["dfjz","wsy",37,4,19,34],
    ["dfjz","clf",30,4,18,32],    ["dfjz","stj",25,6,19,34],    ["dfjz","thlgy"
        ,70,8,38,92],
    ["dfjz","tllgz",70,7,39,102],
    ["szzx","zhzy",35,4,22,38],   ["szzx","szbwg",40,4,23,40],  ["szzx","pjl",33,4,18,32],
    ["szzx","szl",37,4,21,36],    ["szzx","clq",23,3,14,26],    ["szzx","hss",27,6,24,42],
    ["szzx","fqjq",30,6,25,44],   ["szzx","hq",43,6,28,48],     ["szzx","ly",36,4,23,40],
    ["szzx","xyj",32,4,22,38],    ["szzx","wsy",37,4,19,34],    ["szzx","clf",30,4,18,32],
    ["szzx","stj",25,6,19,34],    ["szzx","thlgy",70,8,38,92],  ["szzx","tllgz"
        ,70,7,39,102],
    ["zhzy","szbwg",3,0,3,12],    ["zhzy","pjl",12,0,4,12],     ["zhzy","szl",8,0,3,12],
    ["zhzy","clq",18,0,5,12],     ["zhzy","hss",32,2,14,24],    ["zhzy","fqjq"
        ,36,2,15,26],
    ["zhzy","hq",28,2,13,22],     ["zhzy","ly",24,2,9,16],      ["zhzy","xyj",28,2,9,16],
    ["zhzy","wsy",22,2,9,16],     ["zhzy","clf",22,2,9,16],     ["zhzy","stj",20,2,8,14],
    ["zhzy","thlgy",65,5,33,78],  ["zhzy","tllgz",55,6,34,88],
    ["szbwg","pjl",10,0,4,12],    ["szbwg","szl",7,0,3,12],     ["szbwg","clq",17,0,5,12],
    ["szbwg","hss",33,2,14,24],   ["szbwg","fqjq",37,2,15,26],  ["szbwg","hq",29,2,13,22],
    ["szbwg","ly",25,2,9,16],     ["szbwg","xyj",29,2,9,16],    ["szbwg","wsy",23,2,9,16],
    ["szbwg","clf",23,2,9,16],    ["szbwg","stj",21,2,8,14],    ["szbwg","thlgy"
        ,66,5,33,78],
    ["szbwg","tllgz",56,6,34,88],
    ["pjl","szl",6,0,3,12],       ["pjl","clq",14,0,0,0],       ["pjl","hss",26,6,14,24],
    ["pjl","fqjq",29,6,15,26],    ["pjl","hq",28,7,13,22],      ["pjl","ly",32,4,9,16],
    ["pjl","xyj",28,4,9,16],      ["pjl","wsy",11,0,4,12],      ["pjl","clf",16,0,5,12],
    ["pjl","stj",20,5,8,14],      ["pjl","thlgy",69,8,33,78],   ["pjl","tllgz"
        ,50,6,34,88],
    ["szl","clq",13,0,4,12],      ["szl","hss",31,2,14,24],     ["szl","fqjq",35,2,15,26],
    ["szl","hq",27,2,13,22],      ["szl","ly",23,2,9,16],       ["szl","xyj",27,2,9,16],
    ["szl","wsy",21,2,8,14],      ["szl","clf",21,2,8,14],      ["szl","stj",19,2,8,14],
    ["szl","thlgy",64,5,33,78],   ["szl","tllgz",54,6,34,88],
    ["clq","hss",24,5,11,18],     ["clq","fqjq",27,5,12,20],    ["clq","hq",27,7,11,18],
    ["clq","ly",30,3,9,16],       ["clq","xyj",26,3,9,16],      ["clq","wsy",18,2,4,12],
    ["clq","clf",13,0,4,12],      ["clq","stj",19,5,7,14],      ["clq","thlgy"
        ,68,7,32,76],
    ["clq","tllgz",40,3,33,86],
    ["hss","fqjq",9,0,3,12],      ["hss","hq",23,2,9,16],       ["hss","ly",17,2,4,12],
    ["hss","xyj",13,2,4,12],      ["hss","wsy",33,4,14,24],     ["hss","clf",26,4,14,24],
    ["hss","stj",17,2,8,14],      ["hss","thlgy",55,3,23,54],   ["hss","tllgz"
        ,57,6,32,86],
    ["fqjq","hq",23,2,9,16],      ["fqjq","ly",17,2,4,12],      ["fqjq","xyj",13,2,4,12],
    ["fqjq","wsy",33,4,14,24],    ["fqjq","clf",26,4,14,24],    ["fqjq","stj",17,2,8,14],
    ["fqjq","thlgy",55,3,23,54],  ["fqjq","tllgz",57,6,32,86],
    ["hq","ly",17,2,8,14],        ["hq","xyj",21,2,8,14],       ["hq","wsy",29,4,14,24],
    ["hq","clf",29,4,14,24],      ["hq","stj",13,2,4,12],       ["hq","thlgy",45,3,19,46],
    ["hq","tllgz",57,6,31,82],
    ["ly","xyj",9,0,3,12],        ["ly","wsy",25,4,13,22],      ["ly","clf",25,4,13,22],
    ["ly","stj",13,2,4,12],       ["ly","thlgy",45,3,19,46],    ["ly","tllgz",52,6,31,82],
    ["xyj","wsy",25,4,13,22],     ["xyj","clf",25,4,13,22],     ["xyj","stj",13,2,4,12],
    ["xyj","thlgy",42,3,19,46],   ["xyj","tllgz",52,6,31,82],
    ["wsy","clf",9,0,4,12],       ["wsy","stj",27,4,13,22],     ["wsy","thlgy"
        ,67,7,32,76],
    ["wsy","tllgz",48,5,29,78],
    ["clf","stj",27,4,13,22],     ["clf","thlgy",67,7,32,76],   ["clf","tllgz"
        ,48,5,29,78],
    ["stj","thlgy",55,3,22,52],   ["stj","tllgz",47,5,31,82],
    ["thlgy","tllgz",75,8,38,98],
```

```
    ["szz","dfjz",40,4,25,35],    ["szz","szzx",40,4,25,35],    ["szz","zhzy",25,3,15,20],
    ["szz","szbwg",25,3,15,20],   ["szz","pjl",30,3,12,18],     ["szz","szl",25,3,15,20],
    ["szz","clq",10,2,8,15],      ["szz","hss",35,4,20,30],     ["szz","fqjq",35,4,20,30],
    ["szz","hq",30,3,18,28],      ["szz","ly",20,3,15,22],      ["szz","xyj",20,3,15,22],
    ["szz","wsy",35,4,18,28],     ["szz","clf",25,3,15,25],     ["szz","stj",10,2,8,15],
    ["szz","thlgy",70,6,40,85],   ["szz","tllgz",60,5,45,95],
]

TRAFFIC_MAP = {}
for _r in RAW_TRAFFIC:
    _a, _b, _pt, _pc, _tt, _tc = _r
    _info = {"public_time": int(_pt), "public_cost": int(_pc),
             "taxi_time":   int(_tt), "taxi_cost":   int(_tc)}
    TRAFFIC_MAP[f"{_a}|{_b}"] = _info
    TRAFFIC_MAP[f"{_b}|{_a}"] = _info

# ============================================================
# 3. Utility Functions
# ============================================================
def time_to_min(t):
    """Convert 'HH:MM' string to minutes from midnight (int)."""
    if not t:
        return 0
    h, m = map(int, t.split(":"))
    return h * 60 + m

def min_to_time(m):
    """Convert minutes from midnight to 'HH:MM' string."""
    return f"{(int(m)//60)%24:02d}:{int(m)%60:02d}"

def get_travel(fr, to, mode="public"):
    """
    Return a transport leg dict: {mode, time(min), cost(CNY)}.
    mode="public" -> public transit; mode="taxi" -> Taxi.
    Falls back to {45min, CNY5 public} / {30min, CNY40 taxi} if pair not in matrix.
    """
    if fr == to:
        return {"mode": "stay", "time": 0, "cost": 0}
    info = TRAFFIC_MAP.get(f"{fr}|{to}",
           {"public_time": 45, "public_cost": 5,
            "taxi_time": 30,   "taxi_cost": 40})
    if mode == "taxi":
        return {"mode": "taxi",   "time": info["taxi_time"],   "cost": info["taxi_cost"]}
    return {"mode": "public", "time": info["public_time"], "cost": info["public_cost"]}

def ticket_cost(att, vtype="adult"):
    """Return ticket price for the given visitor type."""
    return att["ticket"].get(vtype, att["ticket"]["adult"])

def selected_preference_types(prefs):
    """Return the set of attraction types that are switched on by w2/w3/w4."""
    types = set()
    if int(prefs.get("w2", 0)) == 1: types.update(HUMANISTIC_TYPES)
    if int(prefs.get("w3", 0)) == 1: types.update(NATURAL_TYPES)
    if int(prefs.get("w4", 0)) == 1: types.update(COMMERCIAL_TYPES)
    return types

def adjusted_score(att, prefs):
    """
    Return s'_i as defined in Eq. (3)-(4):

      Let P be the set of attractions whose types are selected by the user.
      If P = empty, no preference adjustment is applied: s'_i = s_i for all i.
      If P != empty, compute the preference multiplier lambda:

```

```
        lambda = 1.1,                                if max{s_j : j not in P} <= min{s_j : j
             in P}
        lambda = (max{s_j : j not in P} + 1)
                 / min{s_j : j in P},                if max{s_j : j not in P} >  min{s_j : j
                      in P}

      Then:
        s'_i = lambda * s_i,   if i in P,
        s'_i = s_i,            otherwise.
    """
    selected = selected_preference_types(prefs)
    if not selected or att["type"] not in selected:
        return att["score"]

    in_P     = [a["score"] for a in SUZHOU_ATTRACTIONS if a["type"] in selected]
    not_in_P = [a["score"] for a in SUZHOU_ATTRACTIONS if a["type"] not in selected]

    if not not_in_P:
        # All attractions are preferred; no non-preferred set exists
        return att["score"]

    max_not_P = max(not_in_P)
    min_P     = min(in_P)

    if max_not_P <= min_P:
        lam = 1.1
    else:
        lam = (max_not_P + 1) / min_P

    return lam * att["score"]

def route_traffic_time(route, leg_modes, transport="public"):
    """
    Compute total transit-only time (no visit durations) for the full loop
    szz -> route[0] -> ... -> route[-1] -> szz, using per-leg modes when provided.
    leg_modes: list of mode strings, length = len(route)+1 (includes return leg).
    """
    ids   = ["szz"] + [a["id"] for a in route] + ["szz"]
    total = 0
    for i in range(len(ids) - 1):
        mode = leg_modes[i] if (leg_modes and i < len(leg_modes)) else transport
        total += get_travel(ids[i], ids[i + 1], mode)["time"]
    return total


def evaluate(route, day_cfg, budget, prefs,
             must_ids, excl_ids,
             leg_modes=None,
             transport="public",
             vtype="adult"):
    """
    Simulate the trip and return Z plus schedule details.

    Parameters
    ----------
    route      : ordered list of attraction dicts to visit
    day_cfg    : {"startTime": "HH:MM", "endTime": "HH:MM"}
    budget     : float - total budget in CNY (tickets + transport)
    prefs      : {"w2": int, "w3": int, "w4": int,
                  "t_con": float, "t_rest": float}
    must_ids   : set of mandatory attraction IDs
    excl_ids   : set of excluded attraction IDs
    leg_modes  : list of mode strings, one per leg (len = len(route)+1).
                 If None, all legs use `transport`.
    transport  : fallback mode when leg_modes is None
    vtype      : visitor type for ticket pricing
```

```
    Returns
    -------
    dict: Z, feasible, steps, totalCost, visitedAttractions,
          objectiveValue, returnTime, legModes
    """
    t_con  = prefs.get("t_con",  100)
    t_rest = prefs.get("t_rest", 20)

    start_mn = time_to_min(day_cfg["startTime"])
    end_mn   = time_to_min(day_cfg["endTime"])

    n_legs     = len(route) + 1
    used_modes = leg_modes[:] if leg_modes else ["public"] * n_legs

    total_cost = 0.0
    visited    = set()
    cur_time   = start_mn
    cur_loc    = "szz"
    steps      = []
    stamina    = 0.0
    sum_score  = 0.0
    leg_idx    = 0

    for att in route:
        # C1: skip excluded attractions
        if att["id"] in excl_ids:
            break

        mode  = used_modes[leg_idx] if leg_idx < len(used_modes) else transport
        leg   = get_travel(cur_loc, att["id"], mode)
        t_leg = leg["time"]

        # C8: rest before transit if stamina limit would be exceeded
        if stamina + t_leg > t_con:
            cur_time += t_rest
            stamina   = 0.0

        arrival_raw  = cur_time + t_leg
        open_mn      = time_to_min(att["openTime"])
        close_mn     = time_to_min(att["closeTime"])
        wait_time    = max(0, open_mn - arrival_raw)
        arrival_time = arrival_raw + wait_time
        act_dur      = att["baseTime"]
        depart       = arrival_time + act_dur

        # C2b: cannot finish visit before closing time
        if close_mn < 1440 and depart > close_mn:
            break

        sum_score += adjusted_score(att, prefs)

        tcost = ticket_cost(att, vtype)
        total_cost += tcost + leg["cost"]
        visited.add(att["id"])

        # C8: update stamina; reset if continuous limit reached after visit
        stamina += t_leg + act_dur
        if stamina >= t_con:
            cur_time = depart + t_rest
            stamina  = 0.0
        else:
            cur_time = depart
        cur_loc = att["id"]
        leg_idx += 1
```

```
        steps.append({
            "attraction": att,
            "leg":        leg,
            "arriveTime": min_to_time(arrival_time),
            "leaveTime":  min_to_time(depart),
            "waitTime":   wait_time,
            "visitTime":  act_dur,
            "ticketCost": tcost,
        })

    # Return trip to Suzhou Station
    ret_mode  = used_modes[leg_idx] if leg_idx < len(used_modes) else transport
    ret_leg   = get_travel(cur_loc, "szz", ret_mode)
    t_ret_leg = ret_leg["time"]
    if stamina + t_ret_leg > t_con:
        cur_time += t_rest
    ret_time   = cur_time + t_ret_leg
    total_cost += ret_leg["cost"]

    # Objective: Z = sum of adjusted scores (NO penalty deduction)
    unvisited_must = [i for i in must_ids if i not in visited]
    Z = sum_score

    feasible = (len(unvisited_must) == 0
                and ret_time <= end_mn
                and total_cost <= budget)

    return {
        "Z":                  Z,
        "feasible":           feasible,
        "steps":              steps,
        "totalCost":          total_cost,
        "visitedAttractions": list(visited),
        "returnTime":         min_to_time(ret_time),
        "objectiveValue":     round(Z, 4),
        "legModes":           used_modes,
    }

# ============================================================
# 5. Hard-Constraint Check (C1: exclusion, no duplicates)
# ============================================================
def is_hard_feasible(route, excl_ids):
    """Return True iff route contains no excluded or duplicate attractions."""
    seen = set()
    for att in route:
        if att["id"] in excl_ids:
            return False
        if att["id"] in seen:
            return False
        seen.add(att["id"])
    return True

# ============================================================
# 6.Initial Route Construction  (Step 1 of SMMC-SA)
# ============================================================
def _eval_time_cost(route, day_cfg, budget, prefs,
                    must_ids, excl_ids, leg_modes, vtype):
    """Quick helper: evaluate and return (ret_time_min, total_cost)."""
    res = evaluate(route, day_cfg, budget, prefs,
                   must_ids, excl_ids, leg_modes, "public", vtype)
    return time_to_min(res["returnTime"]), res["totalCost"]

def build_initial_route(must_ids, excl_ids, prefs, day_cfg, budget,
                        transport="public", vtype="adult"):
    """
    Construct the initial candidate set and route.
```

```
    Returns
    -------
    (route, leg_modes) or (None, None) if infeasible.
    """
    end_mn = time_to_min(day_cfg["endTime"])

    available_sorted = sorted(
        [a for a in SUZHOU_ATTRACTIONS if a["id"] not in excl_ids],
        key=lambda a: adjusted_score(a, prefs),
        reverse=True
    )

    # Build candidate set C0
    target_size = max(4, len(must_ids))
    route = [a for a in available_sorted if a["id"] in must_ids]
    slots_left = target_size - len(route)
    for att in available_sorted:
        if slots_left <= 0:
            break
        if att["id"] in must_ids:
            continue
        route.append(att)
        slots_left -= 1

    # Phase A: sort by adjusted score
    route = sorted(route, key=lambda a: adjusted_score(a, prefs), reverse=True)

    # Phase B: trim non-mandatory attractions until time is feasible
    leg_modes = ["public"] * (len(route) + 1)
    ret_time, total_cost = _eval_time_cost(route, day_cfg, budget, prefs,
                                           must_ids, excl_ids, leg_modes, vtype)
    while ret_time > end_mn:
        non_must = [a for a in route if a["id"] not in must_ids]
        if not non_must:
            break
        to_drop = min(non_must, key=lambda a: adjusted_score(a, prefs))
        route.remove(to_drop)
        leg_modes = ["public"] * (len(route) + 1)
        ret_time, total_cost = _eval_time_cost(route, day_cfg, budget, prefs,
                                               must_ids, excl_ids, leg_modes, vtype)

    # Phase D: taxi upgrade if still over time after deleting all optionals
    while ret_time > end_mn:
        ids = ["szz"] + [a["id"] for a in route] + ["szz"]
        best_idx = -1
        best_delta_c = float("inf")

        for i in range(len(leg_modes)):
            if leg_modes[i] == "taxi":
                continue
            info = TRAFFIC_MAP.get(f"{ids[i]}|{ids[i+1]}",
                   {"public_time": 45, "public_cost": 5,
                    "taxi_time": 30, "taxi_cost": 40})
            time_saved = info["public_time"] - info["taxi_time"]
            delta_c = info["taxi_cost"] - info["public_cost"]
            if time_saved > 0 and delta_c < best_delta_c:
                best_delta_c = delta_c
                best_idx = i

        if best_idx == -1:
            return None, None

        leg_modes[best_idx] = "taxi"
        ret_time, total_cost = _eval_time_cost(route, day_cfg, budget, prefs,
                                               must_ids, excl_ids, leg_modes, vtype)
```

```
        if total_cost > budget:
            return None, None

    # Phase C: after time is feasible, repair budget by deleting high-ticket optionals
    while total_cost > budget:
        non_must = [a for a in route if a["id"] not in must_ids]
        if not non_must:
            return None, None
        to_drop = max(non_must, key=lambda a: ticket_cost(a, vtype))
        drop_idx = route.index(to_drop)
        route.remove(to_drop)
        if drop_idx < len(leg_modes):
            leg_modes.pop(drop_idx)
        while len(leg_modes) < len(route) + 1:
            leg_modes.append("public")
        while len(leg_modes) > len(route) + 1:
            leg_modes.pop()
        ret_time, total_cost = _eval_time_cost(route, day_cfg, budget, prefs,
                                               must_ids, excl_ids, leg_modes, vtype)
        if ret_time > end_mn:
            return None, None

    return route, leg_modes

# ============================================================
# 7. B&B Exact Ordering  (Step 2 of SMMC-SA)
# ============================================================
def bb_order(candidate_set, day_cfg, budget, prefs,
             must_ids, excl_ids, leg_modes_init, vtype):
    """
    B&B exact ordering for a small candidate set.

    Returns
    -------
    (best_route, best_Z) : (list of att dicts, float)
    """
    adj_scores = {a["id"]: adjusted_score(a, prefs) for a in candidate_set}

    def ev(r):
        modes = ["public"] * (len(r) + 1)
        return evaluate(r, day_cfg, budget, prefs,
                        must_ids, excl_ids, modes, "public", vtype)

    best = {"route": [], "Z": -math.inf}

    def dfs(path, remaining, path_score):
        rem_scores = sorted([adj_scores[a["id"]] for a in remaining], reverse=True)
        ub = path_score + sum(rem_scores)
        if ub <= best["Z"]:
            return

        if not remaining:
            result = ev(path)
            if result["Z"] > best["Z"]:
                best["Z"]     = result["Z"]
                best["route"] = path[:]
            return

        for i, att in enumerate(remaining):
            next_remaining = remaining[:i] + remaining[i+1:]
            dfs(path + [att], next_remaining,
                path_score + adj_scores[att["id"]])

    dfs([], candidate_set[:], 0.0)
    return best["route"], best["Z"]

```

```python
# ============================================================
# 8. Neighbourhood Operators  (used in SA main loop)
# ============================================================
def neighbour(route, leg_modes, available, must_ids, excl_ids):
    """
    Apply one randomly chosen neighbourhood operator.

    Returns
    -------
    (new_route, new_leg_modes)
    """
    new_r     = route[:]
    new_modes = leg_modes[:] if leg_modes else ["public"] * (len(route) + 1)
    op        = random.random()

    if op < 0.20:
        # N1: Transport swap
        if new_modes:
            idx = random.randrange(len(new_modes))
            new_modes[idx] = "taxi" if new_modes[idx] == "public" else "public"

    elif op < 0.50:
        # N2: Insert
        used       = {a["id"] for a in new_r}
        candidates = [a for a in available if a["id"] not in used]
        if candidates:
            att      = random.choice(candidates)
            pos      = random.randint(0, len(new_r))
            new_r.insert(pos, att)
            new_modes.insert(pos, "public")

    elif op < 0.70:
        # N3: 2-opt reversal with C9 anti-detour check
        if len(new_r) >= 2:
            i1, i2 = sorted(random.sample(range(len(new_r)), 2))
            new_r[i1:i2 + 1] = new_r[i1:i2 + 1][::-1]
            new_modes[i1:i2 + 1] = new_modes[i1:i2 + 1][::-1]
            # C9: reject if total travel time increases
            if (route_traffic_time(new_r, new_modes) >
                    route_traffic_time(route, leg_modes)):
                return route[:], leg_modes[:]

    elif op < 0.85:
        # N4: Relocate
        if new_r:
            idx = random.randrange(len(new_r))
            att = new_r.pop(idx)
            new_modes.pop(idx)
            new_pos = random.randint(0, len(new_r))
            new_r.insert(new_pos, att)
            new_modes.insert(new_pos, "public")

    else:
        # N5: Replace non-mandatory attraction with an unselected one
        non_must = [i for i, a in enumerate(new_r) if a["id"] not in must_ids]
        if non_must:
            idx  = random.choice(non_must)
            used = {a["id"] for a in new_r}
            opts = [a for a in available if a["id"] not in used]
            if opts:
                new_r[idx] = random.choice(opts)

    # Ensure leg_modes length stays consistent with route length
    expected = len(new_r) + 1
    while len(new_modes) < expected:
        new_modes.append("public")
```

```
    while len(new_modes) > expected:
        new_modes.pop()

    return new_r, new_modes

# ============================================================
# 9. Main SMMC-SA Solver
# ============================================================
def _run_sa_ri_once(day_cfg, budget, must_visit, excluded,
                    prefs, transport="public", vtype="adult",
                    T0=800.0, T_min=0.5, alpha=0.93, L=40,
                    verbose=True):
    """
    Single run of SMMC-SA.
    """
    t_start   = time.time()
    must_set  = set(must_visit)
    excl_set  = set(excluded)
    available = [a for a in SUZHOU_ATTRACTIONS if a["id"] not in excl_set]

    def ev(r, modes):
        return evaluate(r, day_cfg, budget, prefs,
                        must_set, excl_set, modes, transport, vtype)

    # Step 1: Initial route
    if verbose:
        print(f"\n{'='*60}")
        print("  SMMC-SA - Score-Maximizing Multi-Constraint Simulated Annealing")
        print(f"  T0={T0}  T_min={T_min}  alpha={alpha}  L={L}")
        print(f"  Prefs: humanistic(w2)={prefs.get('w2',0)}  "
              f"natural(w3)={prefs.get('w3',0)}  "
              f"commercial(w4)={prefs.get('w4',0)}")
        print(f"  |M|={len(must_set)}")
        print(f"{'='*60}")
        print("  Step 1: Initial route construction...")

    cur_route, cur_modes = build_initial_route(
        must_set, excl_set, prefs, day_cfg, budget, transport, vtype)

    if cur_route is None:
        if verbose:
            print("  Infeasible: cannot build a valid initial route.")
        return {
            "status":      "infeasible",
            "message":     "infeasible",
            "Z":           "infeasible",
            "feasible":    False,
            "steps":       [],
            "totalCost":   0,
            "visitedAttractions": [],
            "returnTime":  day_cfg["endTime"],
            "objectiveValue": "infeasible",
            "solver":      "SMMC-SA (infeasible)",
            "solveTime":   0,
            "route":       [],
            "days":        [],
            "legModes":    [],
        }

    # Step 2: B&B exact ordering of the initial candidate set
    if verbose:
        print(f"  Route ({len(cur_route)} attractions): "
              f"{', '.join(a['name'] for a in cur_route)}")
        print("  Step 2: B&B exact ordering...")

    cur_route, cur_Z = bb_order(cur_route, day_cfg, budget, prefs,
```

```
                                must_set, excl_set, cur_modes, vtype)
    cur_modes = ["public"] * (len(cur_route) + 1)
    cur_res   = ev(cur_route, cur_modes)

    best_Z     = cur_res["Z"]
    best_route = cur_route[:]
    best_modes = cur_modes[:]
    best_res   = cur_res

    if verbose:
        print(f"  Initial Z = {round(best_Z, 4)}  "
              f"(feasible={cur_res['feasible']})")

    # Step 3: SA main loop
    if verbose:
        print("  Step 3: SA main loop...")

    T              = T0
    total_iters    = 0
    accepted_bad   = 0
    temp_step      = 0
    no_improve     = 0
    NO_IMPROVE_MAX = 50
    prev_best_Z    = best_Z
    cur_Z          = cur_res["Z"]

    while T > T_min:
        temp_step += 1
        for _ in range(L):
            total_iters += 1
            new_r, new_modes = neighbour(
                cur_route, cur_modes, available, must_set, excl_set)

            if not is_hard_feasible(new_r, excl_set):
                continue

            new_res = ev(new_r, new_modes)
            new_Z   = new_res["Z"]
            delta_Z = new_Z - cur_Z

            # Metropolis acceptance criterion
            if delta_Z >= 0:
                cur_route = new_r;  cur_modes = new_modes
                cur_Z     = new_Z;  cur_res   = new_res
            elif random.random() < math.exp(delta_Z / max(T, 1e-9)):
                cur_route = new_r;  cur_modes = new_modes
                cur_Z     = new_Z;  cur_res   = new_res
                accepted_bad += 1

            # Update global best (feasible solutions only)
            if cur_Z > best_Z and cur_res["feasible"]:
                best_Z     = cur_Z
                best_route = cur_route[:]
                best_modes = cur_modes[:]
                best_res   = cur_res

        # Early-stop convergence check
        if best_Z > prev_best_Z + 1e-6:
            prev_best_Z = best_Z
            no_improve  = 0
        else:
            no_improve += 1

        if verbose and temp_step % 20 == 0:
            print(f"  T={T:.2f}  best_Z={round(best_Z,4)}  "
                  f"no_improve={no_improve}/{NO_IMPROVE_MAX}  "
```

```
                  f"accepted_bad={accepted_bad}")

        if no_improve >= NO_IMPROVE_MAX:
            if verbose:
                print(f"  Converged: no improvement for {NO_IMPROVE_MAX} temperature
                    stages "
                      f"(T={T:.4f}), stopping early.")
            break

        T *= alpha   # geometric cooling

    # Final evaluation on the best found route
    final   = ev(best_route, best_modes)
    elapsed = round((time.time() - t_start) * 1000)

    if verbose:
        print(f"\n{'='*60}")
        print(f"  RESULT:  Z = {final['objectiveValue']}")
        print(f"  Feasible:  {final['feasible']}")
        print(f"  Total cost: CNY {final['totalCost']}")
        print(f"  Itinerary timeline:")
        print(f"    {day_cfg['startTime']}  Suzhou Station  [Depart]")
        for s in final["steps"]:
            att  = s["attraction"]
            mode = s["leg"]["mode"]
            print(f"      -> {mode} {s['leg']['time']}min  CNY{s['leg']['cost']}")
            print(f"    {s['arriveTime']}-{s['leaveTime']}  "
                  f"{att['name']:<30}  [{att['type']}]  "
                  f"ticket CNY{s['ticketCost']}")
        if final["steps"]:
            last_id = final["steps"][-1]["attraction"]["id"]
            last_mode = final.get("legModes", ["public"])[-1]
            ret_leg  = get_travel(last_id, "szz", last_mode)
            print(f"      -> {ret_leg['mode']} {ret_leg['time']}min  CNY{ret_leg['cost']}"
                )
        print(f"    {final['returnTime']}  Suzhou Station  [Return]")
        print(f"  Solve time: {elapsed} ms")
        print(f"{'='*60}\n")

    final["solver"]    = "SMMC-SA (Score-Maximizing Multi-Constraint Simulated Annealing)"
    final["solveTime"] = elapsed
    final["route"]     = [s["attraction"]["name"] for s in final["steps"]]
    final["days"]      = [{
        "day":       1,
        "startTime": day_cfg["startTime"],
        "endTime":   day_cfg["endTime"],
        "stops":     final["steps"],
        "dayCost":   final["totalCost"],
    }]
    final["legModes"] = best_modes

    return final


def solve_sa_ri(day_cfg, budget, must_visit, excluded,
                prefs, transport="public", vtype="adult",
                T0=800.0, T_min=0.5, alpha=0.93, L=40,
                verbose=True, restarts=30, seed=None):
    """Run SMMC-SA multiple times and return the feasible solution with the largest Z."""
    t_start = time.time()
    best = None

    actual_seed = seed if seed is not None else random.randint(0, 10**9)

    for k in range(restarts):
        random.seed(actual_seed ^ (k * 0x9E3779B9 & 0xFFFFFFFF))
```

```
        result = _run_sa_ri_once(
            day_cfg=day_cfg, budget=budget,
            must_visit=must_visit, excluded=excluded,
            prefs=prefs, transport=transport, vtype=vtype,
            T0=T0, T_min=T_min, alpha=alpha, L=L,
            verbose=False)
        # Prefer feasible numeric solutions. If all restarts are infeasible,
        # keep an infeasible result and report it explicitly as "infeasible".
        def _score_for_compare(r):
            if not r.get("feasible", False):
                return -math.inf
            z = r.get("objectiveValue", -math.inf)
            return z if isinstance(z, (int, float)) else -math.inf

        if best is None or _score_for_compare(result) > _score_for_compare(best):
            best = result

    random.seed()

    best["solver"] = f"SMMC-SA (best of {restarts} restarts, seed={actual_seed})"
    best["seed"]   = actual_seed
    best["solveTime"] = round((time.time() - t_start) * 1000)

    if verbose:
        print(f"\n{'='*60}")
        print(f"  SMMC-SA finished {restarts} restarts  (seed={actual_seed})")
        print(f"  Best Z = {best['objectiveValue']}  Feasible = {best['feasible']}  Cost =
            CNY{best['totalCost']}")
        print(f"  Solve time: {best['solveTime']} ms")
        _print_route(best, day_cfg)
        print(f"{'='*60}\n")

    return best


def _print_route(result, day_cfg):
    """Print a formatted full route including Suzhou Station start/end."""
    steps = result.get("steps", [])
    print(f"\n  -- Full Route ------------------------------------------------")
    print(f"    {day_cfg['startTime']}  Suzhou Station  [Depart]")
    for s in steps:
        att  = s["attraction"]
        leg  = s["leg"]
        mode_str = "Bus/Metro/Walking" if leg["mode"] == "public" else "Taxi"
        print(f"      -> {mode_str}  {leg['time']}min  CNY{leg['cost']}")
        print(f"    {s['arriveTime']}-{s['leaveTime']}  "
              f"{att['name']:<30}  [{att['type']}]  "
              f"ticket CNY{s['ticketCost']}  score:{att['score']}")
    if steps:
        last_id   = steps[-1]["attraction"]["id"]
        last_mode = result.get("legModes", ["public"] * (len(steps) + 1))[len(steps)]
        ret_leg   = get_travel(last_id, "szz", last_mode)
        mode_str  = "Bus/Metro/Walking" if ret_leg["mode"] == "public" else "Taxi"
        print(f"      -> {mode_str}  {ret_leg['time']}min  CNY{ret_leg['cost']}")
    print(f"    {result['returnTime']}  Suzhou Station  [Return]")
    print(f"  -- Total cost CNY{result['totalCost']}  Z={result['objectiveValue']} ------"
        )

# ============================================================
# 10. Sensitivity Analysis
# ============================================================
SENSITIVITY_DIMENSIONS = {
    "sa_T0": {
        "label":  "Initial Temperature T0",
        "group":  "SA Parameters",
        "values": [50, 200, 500, 800, 1200, 2000],
```

```
        "fmt":    lambda v: str(v),
    },
    "sa_alpha": {
        "label":  "Cooling Coefficient alpha",
        "group":  "SA Parameters",
        "values": [0.70, 0.80, 0.85, 0.90, 0.93, 0.95, 0.98],
        "fmt":    lambda v: f"{v:.2f}",
    },
    "sa_L": {
        "label":  "Inner-Loop Steps L",
        "group":  "SA Parameters",
        "values": [10, 20, 30, 40, 60, 80, 100],
        "fmt":    lambda v: str(v),
    },
    "weight_w2": {
        "label":  "Humanistic Preference",
        "group":  "Preference Weights",
        "values": [0, 1],
        "fmt":    lambda v: str(int(v)),
    },
    "weight_w3": {
        "label":  "Natural Scenery Preference",
        "group":  "Preference Weights",
        "values": [0, 1],
        "fmt":    lambda v: str(int(v)),
    },
    "weight_w4": {
        "label":  "Commercial Preference",
        "group":  "Preference Weights",
        "values": [0, 1],
        "fmt":    lambda v: str(int(v)),
    },
    "budget": {
        "label":  "Total Budget (CNY)",
        "group":  "Constraint Parameters",
        "values": list(range(100, 2001, 100)),
        "fmt":    lambda v: f"CNY{v}",
    },
    "t_con": {
        "label":  "Max Continuous Activity T_con",
        "group":  "Constraint Parameters",
        "values": [30, 60, 90, 120, 150, 180, 210, 240, 270, 300],
        "fmt":    lambda v: f"{int(v)}min",
    },
}


def sensitivity_analysis(dimension, day_cfg, budget, must_visit, excluded,
                         prefs, transport="public", vtype="adult",
                         T0=800.0, T_min=0.5, alpha=0.93, L=40, verbose=True,
                         restarts=30, seed=None, plot=True):
    """
    Run SMMC-SA across a sweep of one parameter dimension.

    Returns
    -------
    {"dimension": str, "sweep": [...], "shadow": float}
    """
    if dimension not in SENSITIVITY_DIMENSIONS:
        raise ValueError(f"Unknown dimension '{dimension}'. "
                         f"Valid: {list(SENSITIVITY_DIMENSIONS.keys())}")

    dim_info = SENSITIVITY_DIMENSIONS[dimension]
    values   = dim_info["values"]
    fmt      = dim_info["fmt"]
    sweep    = []
```

```
    base_seed = seed if seed is not None else random.randint(0, 10**9)

    def _run(bgt, pf, sa_T0, sa_alpha, sa_L):
        return solve_sa_ri(day_cfg=day_cfg, budget=bgt,
                           must_visit=must_visit, excluded=excluded,
                           prefs=pf, transport=transport, vtype=vtype,
                           T0=sa_T0, T_min=T_min, alpha=sa_alpha, L=sa_L,
                           verbose=False, restarts=restarts, seed=base_seed)

    if verbose:
        hr("=")
        print(f"  Sensitivity Analysis  -  Dimension: {dim_info['label']}  ({dim_info['
            group']})")
        hr("=")
        print(f"  {'Param':<12}  {'Z':>8}  {'Cost':>8}  {'#Att':>6}")
        hr("-", 44)

    for v in values:
        sweep_budget = budget
        sweep_prefs  = dict(prefs)
        sweep_T0     = T0
        sweep_alpha  = alpha
        sweep_L      = L

        if   dimension == "sa_T0":    sweep_T0 = v
        elif dimension == "sa_alpha": sweep_alpha = v
        elif dimension == "sa_L":     sweep_L = int(v)
        elif dimension == "budget":   sweep_budget = float(v)
        elif dimension == "t_con":    sweep_prefs["t_con"] = float(v)
        elif dimension in {"weight_w2", "weight_w3", "weight_w4"}:
            sweep_prefs[dimension[7:]] = int(v)

        r = _run(sweep_budget, sweep_prefs, sweep_T0, sweep_alpha, sweep_L)
        n = len(r.get("visitedAttractions", []))
        pt = {"param": v if v != float("inf") else 99999,
              "label": fmt(v), "Z": r["objectiveValue"],
              "totalCost": r["totalCost"], "n": n}
        sweep.append(pt)

        if verbose:
            print(f"  {fmt(v):<12}  {r['objectiveValue']:>8.4f}  "
                  f"CNY{r['totalCost']:>6}  {n:>6}")

    # Shadow price: central finite difference at midpoint
    mid    = len(sweep) // 2
    shadow = 0.0
    if len(sweep) >= 3:
        lo    = sweep[max(0, mid - 1)]
        hi    = sweep[min(len(sweep) - 1, mid + 1)]
        denom = hi["param"] - lo["param"]
        if denom != 0:
            shadow = round((hi["Z"] - lo["Z"]) / denom, 4)

    if verbose:
        hr("-", 44)
        print(f"  Midpoint shadow price dZ/d_param = {shadow:+.4f}")
        if shadow > 0:
            print(f"  -> Increasing '{dim_info['label']}' improves Z")
        elif shadow < 0:
            print(f"  -> Increasing '{dim_info['label']}' reduces Z")
        else:
            print(f"  -> This parameter has negligible effect on Z under current config")
        hr("=")

        z_vals  = [pt["Z"] for pt in sweep]
        z_min   = min(z_vals); z_max = max(z_vals)
```

```
        z_range = z_max - z_min if z_max != z_min else 1
        bar_w   = 30
        print(f"\n  Z Sensitivity Curve  (Y-axis: Z,  X-axis: {dim_info['label']})")
        hr("-", 56)
        for pt in sweep:
            bar_len = int((pt["Z"] - z_min) / z_range * bar_w)
            bar = "#" * bar_len
            print(f"  {fmt(pt['param']):>10}  {bar:<{bar_w}}  {pt['Z']:.4f}")
        hr("-", 56)

        if plot:
            try:
                import matplotlib.pyplot as plt
                xs = [pt["param"] for pt in sweep]
                ys = [pt["Z"]     for pt in sweep]
                fig, ax = plt.subplots(figsize=(7, 4))
                ax.plot(xs, ys, "o-", color="#4f46e5", linewidth=2, markersize=6)
                ax.set_xlabel(dim_info["label"], fontsize=12)
                ax.set_ylabel("Z (Objective Value)", fontsize=12)
                ax.set_title(f"Sensitivity Analysis - {dim_info['label']}  ({restarts}
                    restarts/pt, seed={base_seed})", fontsize=13)
                ax.axhline(y=ys[len(ys)//2], color="gray", linestyle="--", alpha=0.5,
                    label="Midpoint Z")
                ax.grid(True, alpha=0.3)
                ax.legend(fontsize=9)
                plt.tight_layout()
                plt.show()
            except ImportError:
                print("  (matplotlib not installed, skipping chart)")

    return {"dimension": dimension, "sweep": sweep, "shadow": shadow}


# ============================================================
# 11. Interactive CLI
# ============================================================
def hr(char="-", width=64):
    print(char * width)

def section(title):
    print(); hr("="); print(f"  {title}"); hr("=")

def prompt(msg, default=None):
    hint = f"  [{default}]" if default is not None else ""
    val  = input(f"  {msg}{hint}: ").strip()
    return val if val else (str(default) if default is not None else "")

def prompt_float(msg, lo, hi, default):
    while True:
        raw = prompt(msg, default)
        try:
            v = float(raw)
            if lo <= v <= hi: return v
            print(f"  Please enter a value between {lo} and {hi}")
        except ValueError:
            print("  Please enter a number")

def prompt_int(msg, lo, hi, default):
    while True:
        raw = prompt(msg, default)
        try:
            v = int(raw)
            if lo <= v <= hi: return v
            print(f"  Please enter an integer between {lo} and {hi}")
        except ValueError:
            print("  Please enter an integer")
```

```
def choose_from(options, title, default_key):
    print(f"\n  {title}"); hr("-", 48)
    for k, v in options.items():
        marker = " <- default" if k == default_key else ""
        label  = v[1] if len(v) > 1 else v[0]
        desc   = f"  - {v[2]}" if len(v) > 2 else ""
        print(f"    {k}. {label}{desc}{marker}")
    hr("-", 48)
    while True:
        choice = prompt("Select number", default_key)
        if choice in options: return options[choice]
        print(f"  Please enter one of: {'/'.join(options.keys())}")

VISITOR_TYPES = {
    "1": ("adult",   "Adult"),
    "2": ("student", "Student (undergraduate and below)"),
    "3": ("child",   "Child (6-18 years old)"),
    "4": ("senior",  "Senior (60-69 years old)"),
}

TRANSPORT_TYPES = {
    "1": ("public", "Public Transport (default)"),
    "2": ("taxi",   "Taxi / Private Car"),
}


def run_interactive():
    print(); hr("=")
    print("  SMMC-SA - Score-Maximizing Multi-Constraint Simulated Annealing")
    print("  Suzhou One-Day Tour Route Optimization (Interactive CLI)")
    hr("=")

    section("Step 1 - Trip Basic Settings")
    start_time   = prompt("Departure time (HH:MM)", "09:00")
    end_time     = prompt("Latest return time (HH:MM)", "21:00")
    vt_entry     = choose_from(VISITOR_TYPES, "Visitor type (affects ticket prices)", "1")
    visitor_type = vt_entry[0]
    budget       = prompt_float("Total budget (CNY, tickets + transport)", 50, 5000, 300)
    transport    = "public"
    day_cfg      = {"startTime": start_time, "endTime": end_time}

    section("Step 2 - Attraction Selection")
    print("  The algorithm will automatically select the optimal combination from the 17
        attractions below.\n")
    for i, a in enumerate(SUZHOU_ATTRACTIONS, 1):
        tcost  = a["ticket"].get(visitor_type, a["ticket"]["adult"])
        tc_str = "Free" if tcost == 0 else f"CNY{tcost}"
        print(f"  {i:2}. {a['name']:<30} [{a['type']:<12}] "
              f"score:{a['score']}  {a['baseTime']}min  {tc_str}  "
              f"{a['openTime']}-{a['closeTime']}")
    hr("-")

    def parse_ids(raw):
        ids = set()
        for tok in raw.split():
            try:
                n = int(tok)
                if 1 <= n <= len(SUZHOU_ATTRACTIONS):
                    ids.add(SUZHOU_ATTRACTIONS[n - 1]["id"])
            except ValueError:
                pass
        return ids

    must_ids = parse_ids(prompt("Mandatory attraction numbers (space-separated, leave
        blank=none)", ""))
```

```
    excl_ids = parse_ids(prompt("Excluded attraction numbers (space-separated, leave blank
        =none)", "")) - must_ids

    if must_ids:
        print(f"  Mandatory: {', '.join(a['name'] for a in SUZHOU_ATTRACTIONS if a['id']
            in must_ids)}")
    if excl_ids:
        print(f"  Excluded:  {', '.join(a['name'] for a in SUZHOU_ATTRACTIONS if a['id']
            in excl_ids)}")
    if not must_ids and not excl_ids:
        print("  No special restrictions, algorithm selects freely")

    section("Step 3 - Type Preferences (0/1)")
    print("  Enter 1 to prefer that type of attraction, 0 for no preference.\n")
    w2 = prompt_int("Humanistic/historical preference w2", 0, 1, 0)
    w3 = prompt_int("Natural scenery preference w3", 0, 1, 0)
    w4 = prompt_int("Commercial/leisure preference w4", 0, 1, 0)

    section("Step 4 - Stamina Constraint C8")
    print("  T_con:  Maximum continuous activity duration (min, 30-300), rest is inserted
        after exceeding this")
    print("  T_rest: Mandatory rest duration (min)\n")
    t_con  = prompt_float("T_con max continuous activity (30-300 min)", 30, 300, 100)
    t_rest = prompt_float("T_rest mandatory rest duration", 5, 60, 20)

    prefs = {"w2": w2, "w3": w3, "w4": w4, "t_con": t_con, "t_rest": t_rest}

    section("Step 4.5 - Random Seed")
    print("  Setting a seed makes all 30-restart results fully reproducible.")
    print("  Leave blank for random (non-reproducible).\n")
    seed_raw = prompt("Random seed (integer, blank=random)", "")
    run_seed = int(seed_raw) if seed_raw.strip().isdigit() else None
    if run_seed is not None:
        print(f"  Seed set = {run_seed} (results are reproducible)")
    else:
        print("  Random seed (results may vary each run)")

    section("Step 5 - SA Hyperparameters (press Enter to use defaults)")
    print(f"  {'Parameter':<30}  {'Default':>6}  Description"); hr("-", 60)
    print(f"  {'T0  Initial temperature':<30}  {'800.0':>6}  Higher = more likely to
        accept worse solutions")
    print(f"  {'T_min  Stopping temperature':<30}  {'0.5':>6}  Stop annealing below this
        value")
    print(f"  {'alpha  Cooling coefficient':<30}  {'0.93':>6}  Temperature *= alpha each
        round")
    print(f"  {'L  Inner-loop steps':<30}  {'40':>6}  Markov chain length")
    hr("-", 60)
    T0    = prompt_float("T0  Initial temperature", 10, 5000, 800.0)
    T_min = prompt_float("T_min  Stopping temperature", 0.01, 100, 0.5)
    alpha = prompt_float("alpha  Cooling coefficient", 0.5, 0.999, 0.93)
    L     = prompt_int  ("L  Inner-loop steps", 5, 500, 40)

    print(f"\n  {'-'*62}")
    print("  Configuration summary:")
    print(f"    Time: {start_time} - {end_time}")
    print(f"    Visitor: {visitor_type}  |  Budget: CNY{budget}  |  Transport: {transport}
        ")
    print(f"    humanistic={w2}  natural={w3}  commercial={w4}")
    print(f"    T_con={int(t_con)}min  T_rest={t_rest}min")
    print(f"    Mandatory: {len(must_ids)}  |  Excluded: {len(excl_ids)}")
    print(f"    SA: T0={T0}  T_min={T_min}  alpha={alpha}  L={L}")
    print(f"  {'-'*62}")

    confirm = prompt("Confirm run? (y/n)", "y")
    if confirm.lower() not in ("y", "yes", ""):
        print("  Cancelled."); return
```

```
    result = solve_sa_ri(
        day_cfg=day_cfg, budget=budget,
        must_visit=list(must_ids), excluded=list(excl_ids),
        prefs=prefs, transport=transport, vtype=visitor_type,
        T0=T0, T_min=T_min, alpha=alpha, L=L, verbose=True,
        seed=run_seed)

    print("\n  Done!")

    section("Step 6 - Sensitivity Analysis (optional)")
    groups  = {}
    for did, dinfo in SENSITIVITY_DIMENSIONS.items():
        groups.setdefault(dinfo["group"], []).append((did, dinfo))
    idx     = 1
    dim_map = {}
    for gname, dims in groups.items():
        print(f"  -- {gname} --")
        for did, dinfo in dims:
            print(f"    {idx}. {dinfo['label']}")
            dim_map[str(idx)] = did
            idx += 1
    print("    0. Skip"); hr("-", 48)

    while True:
        choices_raw   = prompt("Select dimension numbers to analyse (space-separated, 0=
            skip)", "0")
        tokens        = choices_raw.split()
        if tokens == ["0"]:
            print("  Sensitivity analysis skipped."); break
        selected_dims = [dim_map[t] for t in tokens if t in dim_map]
        if not selected_dims:
            continue
        use_same    = prompt("Use same SA hyperparameters as above? (y/n)", "y")
        sweep_T0    = T0;    sweep_alpha = alpha;    sweep_L = L
        if use_same.lower() not in ("y", "yes", ""):
            sweep_T0    = prompt_float("T0", 10, 5000, T0)
            sweep_alpha = prompt_float("alpha", 0.5, 0.999, alpha)
            sweep_L     = prompt_int  ("L", 5, 500, L)
        print()
        for dim in selected_dims:
            sensitivity_analysis(
                dimension=dim, day_cfg=day_cfg, budget=budget,
                must_visit=list(must_ids), excluded=list(excl_ids),
                prefs=prefs, transport=transport, vtype=visitor_type,
                T0=sweep_T0, T_min=T_min, alpha=sweep_alpha, L=sweep_L,
                verbose=True, restarts=30, seed=run_seed, plot=True)
        if prompt("\nAnalyse other dimensions? (y/n)", "n").lower() not in ("y", "yes"):
            break

    print("\n  All done!")
    return result


if __name__ == "__main__":
    run_interactive()
```